

Claude Code v2.1.309
Opus 5.5 · Claude Pro
~/Desktop/test work project

Sonnet 5.5 is here, and it's faster and excels at well-scoped tasks. Opus 5.5 is still your default. Switch with /model.

❯ привіт в мене тестове на інтерна (software developer intern). прочитай README.md, index.html і html2json.js і скажи коротко списком що від мене хочуть. хочу спочатку нормально розібратись бо потім буде код рев'ю і треба буде пояснювати

Ran 1 shell command (ctrl+o to expand)

● Коротко, що від тебе хочуть.

Саме завдання

- Реалізувати html2json(htmlText) у html2json.js. Функція приймає HTML як рядок і повертає JS-об'єкт. Зараз там заглушка. Решту файлу (convertHtml2JsonAndSet, showExample1/2) можна лишити як є, вона потрібна для UI в index.html.
- DOM-парсер використовувати не можна. Тобто ніяких DOMParser, innerHTML чи document.createElement. Парсер (токенізатор + побудову дерева) пишеш сам.
- Формат JSON обираєш сам. Його не задали навмисно, перевіряють, як ти вирішуєш задачу без повного ТЗ. Будь готовий пояснити кожне рішення: чому tag/attributes/children, як зберігаються текстові вузли, коментарі, doctype тощо.

Як оцінюватимуть

- Код НЕ має падати на жодному вході. Це головна вимога: якщо валідний HTML ламає функцію, це провал.
- Покриття різних структур і розмірів: від одного тегу до великої сторінки.
- Edge cases, які варто передбачити:
  - void-теги без закриття: <meta>, <link>, <br>, <img>, <input>
  - <!DOCTYPE> і <!-- коментарі -->
  - вміст <script>, <style>, <textarea>. Це raw text, тобто < усередині не є тегом
  - атрибути без значення (disabled), в одинарних, подвійних лапках і без лапок
  - entities (© є прямо в прикладі 1)
  - пробіли й переноси між тегами (whitespace-only текст)
  - невалідний HTML: незакриті теги, зайві , порожній рядок, просто текст без тегів
- Чистота й форматування коду.

Що здати (структура репо)

- html2json.js з реалізацією
- index.html, можна без змін, але обов'язково включити
- html_samples/ з файлами HTML, на яких ти тестував
- ai_help/ з доказами використання AI. Використання AI обов'язкове, і всю історію треба вести в одній розмові:
  - для ChatGPT це chatgpt_chat.txt з посиланням
  - для інших інструментів (як у тебе з Claude Code) треба прикласти .pdf/.png/.mp4 з розмовою. Тож одразу думай, як ти цю сесію експортуватимеш
- README.md за бажанням можна переписати і пояснити там свої рішення. Раджу це зробити, на код рев'ю допоможе.

Перед відправкою

- Перевір, що всі посилання відкриваються в інкогніто. Якщо посилання бите, роботу не дивитимуться.
- Ще раз прожени свої семпли й переконайся, що нічого не падає.
- Перевір, що всі файли на місці. Якщо чогось бракує, роботу зарахують як провалену.
- Якісь важливіші за швидкість.

Дрібниця: README посилається на розділ «Expected Deliverables», але він насправді називається «Expected repository structure». Це одне й те саме.

Можу далі допомогти продумати JSON-структуру і пояснити, чому саме така. Це найкраще покласти на початок, бо на рев'ю саме про неї питатимуть першою.

✻ Cooked for 21s · done Saturday 9:57 PM

❯ давай спочатку git init і перший коміт з оригінальними файлами, щоб якщо шо можна було відкотитись. і створи папки html_samples/ і ai_help/ як в README

  Ran 2 shell commands (ctrl+o to expand)

● Репозиторій створено, перший коміт 7cb7a2a містить оригінальні README.md, index.html і html2json.js. Відкотитись до нього можна так: git checkout 7cb7a2a -- .

  Що ще зробив:
  - Додав .gitignore з .DS_Store, щоб службовий файл macOS не потрапив у репо. Він теж у першому коміті.
  - Створив папки html_samples/ і ai_help/ з порожнім .gitkeep у кожній: git не зберігає порожні папки. Їх я поки не комітив, бо в цьому коміті мали бути лише оригінальні файли. Закомічу разом із першими семплами або окремо, як скажеш.

  Ім'я й email для git не налаштовані, тому git сам підставив автора Barsik <barsik@ip-192-168-0-231.tv9.home>. Якщо репо піде на GitHub або в архів для рев'ю, краще налаштувати нормальні дані:
  ! git config user.name "Твоє Ім'я"
  ! git config user.email "[email removed]"
  ! git commit --amend --reset-author --no-edit
  Останній рядок перепише автора вже зробленого коміту.

  Далі пропоную продумати JSON-структуру.

✻ Baked for 16s · done Saturday 9:59 PM

❯ гаразд з гітом трохи пізніше розберемось, нагадаєш мені про нього в кінці, я тобі ще скажу чи буде це валідно чи я змінив свою думку. так, DOMParser юзати не можна, значить треба писати свій парсер. з чого він взагалі складається? чув шось про токенайзер і дерево, поясни просто як це працює разом. і які взагалі штуки можуть бути в html з якими парсер має впоратись? давай до коду складемо список edge cases. я поки що думаю про void теги, коментарі, doctype, script/style, атрибути без лапок. шо ще?

● Як влаштований парсер

  Парсер складається з двох частин, які працюють одна за одною.

  1. Токенізатор (лексер). Він іде по рядку символ за символом і ріже його на «шматки», тобто токени. Про вкладеність він нічого не знає, просто бачить шматки:

  Hi
  перетворюється на
  StartTag  { name: "div", attrs: { class: "a" } }
  StartTag  { name: "p" }
  Text      { "Hi" }
  EndTag    { name: "p" }
  EndTag    { name: "div" }
  Типи токенів: StartTag, EndTag, Text, Comment, Doctype.

  Усередині це state machine: парсер завжди перебуває в якомусь стані («читаю текст», «читаю ім'я тегу», «читаю значення атрибута в лапках»…), і поточний символ визначає, у який стан перейти далі.

  2. Побудова дерева (tree builder). Він бере токени по черзі й будує вкладену структуру за допомогою стеку відкритих елементів:
  - StartTag: створює вузол, додає його в children того, хто зараз на вершині стеку, і кладе новий вузол у стек (крім void-тегів)
  - Text / Comment: просто додає їх у children вершини стеку
  - EndTag: знімає елементи зі стеку, доки не знайде тег з таким самим ім'ям

  стек: [root]            ← 
  стек: [root, div]       ← 
  стек: [root, div, p]    ← "Hi" йде в p.children
  стек: [root, div]       ← 
  стек: [root]            ← 

  Аналогія: токенізатор розбиває речення на слова, а tree builder розбирає граматику, тобто що до чого належить.

Важливий момент: після <script> або <style> токенізатор має переключитися в режим, де він не шукає теги, а читає все до </script>. Тобто ці дві частини не зовсім незалежні.

---

Edge cases

Твої п'ять пунктів є в списку, решту я додав. ⭐ означає must-have, без цього код або впаде, або дасть явно неправильний результат.

Структура документа

- ⭐ Кілька коренів (фрагмент, як у прикладі 2: a b ). Тому результат має бути або масивом, або вузлом root з children.
- ⭐ Просто текст без тегів, порожній рядок, рядок з самих пробілів.
- <!DOCTYPE html> і <!doctype html>: регістр не має значення.
- BOM на початку файлу, переноси рядків \r\n.

Теги

- ⭐ Void-теги: br, hr, img, input, meta, link, area, base, col, embed, source, track, wbr. Закриваючого тегу в них немає.
- ⭐ Self-closing синтаксис <br/>, <br />. Підступ: у HTML  не закриває div, слеш просто ігнорується. Треба вирішити, чи ти повторюєш поведінку браузера, чи трактуєш це як закритий тег.
- ⭐ Регістр: <DIV>… означає той самий тег.
- Пробіли й переноси всередині тегу: <div\n  class="a"\n>.
- SVG: <path /> там справді self-closing, а в атрибутах важливий регістр (viewBox).
- <template>.

Атрибути

- ⭐ Без значення (disabled), у "подвійних" і 'одинарних' лапках, без лапок (class=a).
- ⭐ > або лапки всередині значення: title="a > b", data-x='він сказав "привіт"'. Наївний regex на > тут зламається.
- Порожнє значення a="". Тут треба вирішити, чи відрізняється воно від a без значення.
- Пробіли навколо =: class = "a".
- Дублікати: <a href="1" href="2">. У браузері перемагає перший.
- Регістр імен: CLASS і class означають одне.
- ⭐ __proto__, constructor як ім'я атрибута. Якщо атрибути складати у звичайний {}, можна отримати prototype pollution або зламаний об'єкт. Рішення: Object.create(null) або масив пар.

Текст і entities

- ⭐ <, яке не є тегом: a < b, 1<2, <3. Тег починається тільки тоді, коли за < іде літера, / або !.
- ⭐ Entities: іменовані (©, &,  ), числові (©), hex (©), без ; (&copy), невідомі (&foo;), самотній &. Треба вирішити, чи декодуєш ти їх, чи лишаєш як є. Повна таблиця містить 2000+ назв, тому реалістично взяти основні й пояснити, чому саме їх.
- ⭐ Whitespace-only текст між тегами (\n  ): зберігати чи викидати? Виняток: у <pre> і <textarea> пробіли значущі.
- Unicode: кирилиця, емодзі.

Raw text елементи

- ⭐ <script>, <style>: усередині немає ні тегів, ні entities, читаєш до </script>. Сюди ж кейс if (a<b) у JS і рядок " " у скрипті.
- ⭐ <textarea>, <title>: тегів усередині немає, але entities декодуються (RCDATA). Пов'язано з прикладом 2.
- </SCRIPT >: закриваючий тег теж без урахування регістру і з пробілом.
- Нішеві: <xmp>, <noscript>, <iframe>, <plaintext>. Можна просто згадати в README.

Коментарі та інше <!...>

- ⭐ <!-- звичайний -->, багаторядковий, порожній <!---->, з тегами всередині <!-- <div> -->.
- ⭐ Незакритий коментар <!-- ... до кінця файлу.
- <![CDATA[...]]>, <?xml ...?>. Браузер трактує їх як коментар.
- Conditional comments <!--[if IE]>...<![endif]-->.

Невалідний HTML (головне для «не падати»)

- ⭐ Незакриті теги: <p>текст до кінця файлу.
- ⭐ Зайвий закриваючий тег , якого не відкривали. Його треба ігнорувати, а не робити pop усього стеку.
- ⭐ Неправильна вкладеність: x.
- ⭐ Обрізаний тег: <div class="a (EOF посеред атрибута), <, </ у самому кінці.
- Неявне закриття: <p>a<p>b, <li>, <td>, <tr>, <option>. Браузер закриває попередній <p>/<li> автоматично. Повний алгоритм HTML5 величезний, тож можна взяти кілька найчастіших правил.
- Браузер ще й сам додає <html>/<head>/<body>. Тут треба вирішити, чи робиш так само. Я б не робив: на виході має бути те, що ввели.

Вхідні дані й розмір

- ⭐ Не рядок на вході: null, undefined, число. Функція має не падати, а повернути щось зрозуміле.
- ⭐ Глибока вкладеність (10 000 ). Рекурсія дасть stack overflow, тому tree builder краще робити ітеративним, на стеку. Він і так на стеку, тож це майже безкоштовно.
- ⭐ Великий вхід (кілька МБ). Уникати O(n²): склеювання рядків у циклі, slice на кожному символі, regex з catastrophic backtracking.
- ⭐ Циклічні посилання. Якщо додаєш вузлу поле parent, JSON.stringify впаде з Converting circular structure to JSON. Це справжній crash в index.html.

---

Частина пунктів стосується не того, як не впасти, а того, що повертати: entities, whitespace, , неявне закриття. Саме ці рішення на рев'ю й питатимуть «чому так». Тому наступний крок я б зробив такий: зафіксувати JSON-структуру і ці рішення, а потім покласти семпли з цього списку в html_samples/, щоб одразу було на чому тестувати.

Про git нагадаю в кінці.

✻ Cooked for 51s · done Saturday 10:08 PM

※ recap: Пишемо свій HTML-парсер (без DOMParser) для тестового завдання html2json: розібрались, як працюють токенізатор і дерево, і склали список edge cases. Далі зафіксувати JSON-структуру й рішення щодо entities та пробілів, потім покласти семпли в html_samples/. (disable recaps in /config)

❯ окей, якщо я правильно зрозумів: токенайзер ріже рядок на токени (StartTag, EndTag, Text, Comment, Doctype), а tree builder через стек збирає з них дерево. і ще є частина рішень, які треба прийняти не "як не впасти", а "що повертати". давай зафіксуємо формат json і ці рішення. я думаю так:
1 повертаємо { type: "root", children: [...] }, бо може бути кілька коренів
2 html/head/body сам не додаємо, повертаємо те що ввели
3 текст де тільки пробіли викидаємо, крім pre і textarea. звичайний текст не тримаємо
з приводу іншого то думаю entities декодуємо основні + числові, невідомі лишаємо як є
 вважаємо закритим
неявне закриття тільки для p, li, option, tr, td/th
атрибут без значення = "", дублікати: перший перемагає
теги і атрибути в нижній регістр
без поля parent, щоб не було циклічних посилань
шо думаєш? де я неправий або шо ще треба вирішити? покажи приклад json для example 2

● Ти все зрозумів правильно, і більшість рішень хороші. Нижче окремо, де я з тобою згоден, а де варто доопрацювати.

Де я згоден

| Рішення | Як обґрунтувати на рев'ю |
|---|---|
| { type: "root", children } | Фрагмент може мати кілька коренів. Плюс у кожного вузла однакова форма, тож код, який обходить дерево, не потребує особливого випадку для кореня |
| Не додаємо html/head/body | Ми конвертуємо те, що ввели, а не емулюємо браузер. На виході нема вузлів, яких не було на вході |

| | |
|---|---|
| Не trim-имо текст | Пробіли на краях тексту бувають значущими (Hello world) |
|  вважаємо закритим | Тут ми свідомо відходимо від браузера: автор явно мав на увазі закритий тег (звичка з JSX/XML). Варто записати в README |
| Атрибут без значення = "" | Браузер робить так само (el.getAttribute("disabled") === "") |
| Дублікати: перший перемагає | Як у браузері |
| Без parent | Інакше JSON.stringify впаде на циклічному посиланні |

Де варто доопрацювати

1. Пробіли між inline-тегами (найважливіше).
Hello world: пробіл між тегами — це якраз whitespace-only текст. Якщо його викинути, вийде «Helloworld», тобто результат стане неправильним.

Пропоную таке правило: викидаємо whitespace-only текст, тільки якщо він містить перенос рядка. Такий текст майже завжди є відступами форматування, а одиночний " " між тегами зберігаємо. Правило просте, легко пояснюється і покриває обидва випадки.

2. Що вважати «пробілом».
Не використовуй /^\s*$/: у JS \s включає \u00A0, тобто  . Тоді <td> </td> після декодування втратить свій вміст. За специфікацією HTML пробіли — це тільки [ \t\n\r\f].

3. Неявне закриття <p>.
<p> закривається не лише наступним <p>, а й будь-яким блочним тегом: <p>текст теж закриває p. Тому для p потрібен список: div, ul, ol, table, h1–h6, section, header, footer, nav, main, pre, blockquote, form, hr, p….

4. Вкладені списки й таблиці.
<ul><li>a<ul><li>b — внутрішній <li> не повинен закривати зовнішній. Найпростіше правило: закриваємо, тільки якщо поточний вузол (вершина стеку) — li. Так ти ніколи не пройдеш крізь вкладений <ul>. Те саме для td/tr/option.

5. Нижній регістр і SVG.
viewBox → viewbox губить інформацію, бо SVG чутливий до регістру. Є два варіанти:
- (а) приводити все до нижнього регістру і записати це як відоме обмеження
- (б) не змінювати регістр усередині <svg>/<math>

Раджу (б): там це одна перевірка.

Що ще треба вирішити

- Форма вузлів. Пропоную element / text / comment / doctype. Поля attributes і children у елемента є завжди, навіть у void-тегів, щоб структура була передбачуваною.
- Атрибути як об'єкт, а не масив. Так зручніше. Створювати через Object.create(null) (захист від __proto__). Нюанс: ключі на кшталт "1" JS переставить на початок, але це дрібниця.
- Де декодувати entities. У тексті й значеннях атрибутів, у textarea/title теж. Не в script/style.
- Вміст script/style — один text-вузол у children.
- Сусідні text-вузли зливаємо. ab (зайвий  ігнорується) дасть один вузол "ab", а не два.
- Закриваючий тег, якого немає в стеку, ігноруємо. Якщо він є, знімаємо зі стеку все до нього. Тоді для x маємо:  закриває обидва,  ігнорується.
- EOF. Незакриті теги закриваємо мовчки, незакритий коментар іде до кінця файлу.
- Не рядок на вході (null, число): повертаємо порожній { type: "root", children: [] }. Можна й String(x), але я б не вгадував, що мали на увазі.
- Помилки. За бажанням можна додати root.warnings: [...] («незакритий », «зайвий »). Буде плюсом на рев'ю, але це не обов'язково, можна лишити на кінець.

Example 2 у цьому форматі

```
{
  "type": "root",
  "children": [
    {
      "type": "element",
      "tag": "div",
      "attributes": {},
      "children": [
        {
          "type": "element",
          "tag": "p",
          "attributes": {},
          "children": [
            { "type": "text", "content": "Hello world!" }
          ]
        },
        {
          "type": "element",
          "tag": "button",
          "attributes": {},
          "children": [
            { "type": "text", "content": "Click me!" }
          ]
        },
        {
          "type": "element",
          "tag": "textarea",
          "attributes": {},
          "children": [
            { "type": "text", "content": "Some very very ... very long
string." }
          ]
        }
      ]
    }
  ]
}
```

Переноси й відступи між , <p>, <button> викинуто (whitespace з \n). Так само викинуто \n після . Текст усередині тегів не обрізано.

Для порівняння, шматок з example 1 з атрибутами й doctype:
{ "type": "doctype", "content": "html" },
{ "type": "element", "tag": "meta", "attributes": { "charset": "UTF-8" },
"children": [] },
{ "type": "element", "tag": "p", "attributes": {}, "children": [
  { "type": "text", "content": "© 2024 My Website" }
]}

Якщо погоджуєшся з пунктами 1–5 і формою вузлів, наступний крок такий: записати ці рішення в README (потім знадобиться на рев'ю) і зробити семпли в html_samples/. Після цього вже пишемо токенізатор.

✻ Crunched for 50s · done Saturday 10:32 PM

❯ окей, по пунктах:
1 правило з переносом рядка норм, але тоді a\nb склеїться в "ab", хоча браузер покаже "a b". як я розумію, це трейдоф, і краще просто записати в обмеження?
2 ок буду юзати [ \t\n\r\f] замість \s. але чому \s взагалі ловить nbsp?
3 ок список блочних тегів для p беремо
4 ок li закриває тільки якщо на вершині стеку li, логічно
5 беру варіант (б), регістр в svg/math не чіпаю

по решті згоден. warnings хочу додати, звучить корисно для дебагу. і поясни нормально про Object.create(null) і __proto__, як саме атрибут може щось зламати?
потім записуємо рішення в README і робимо семпли

також давай будь ласка будемо фіксувати на якому етапі +- завдання ми вже, тобто мати приблизний якийсь роадмеп для себе щоб не губитись і добре проудумавти кожен крок, сповіщай мене кожен раз на якому етапі ми у відсотках чи якщо ти це сплануєш типу там 100 етапів то ми там на етапі номер .. ""

Ran 1 shell command (ctrl+o to expand)

- Спочатку відповіді на питання, потім роадмеп, а далі запишу README і семпли.

  1. a\nb

  Так, це трейдоф, і його треба записати в обмеження. Щоб вирішити його правильно, треба знати, чи елемент inline, чи блочний. А це залежить від CSS (span можна зробити display: block), тому з одного HTML цього в принципі не визначиш. Альтернатива — замінювати такий текст на " " замість викидання. Тоді «a b» буде правильним, але між кожними двома блочними тегами з'явиться сміттєвий вузол " ", і JSON стане значно шумнішим. Для формату, який читають люди й програми, я б лишив викидання і чесно описав обмеження.

  2. Чому \s ловить nbsp

  Перевірив у Node: /\s/.test("\u00A0") дає true. Регулярки в JS — загальний інструмент для Unicode-тексту, і \s там означає «будь-який пробільний символ Unicode»: nbsp, \uFEFF (BOM), \u2000–\u200A, \u3000 (ідеографічний пробіл) та інші. А в HTML своє, вужче визначення: пробіли — це тільки ASCII [ \t\n\r\f].   у HTML — це контент, бо його вставляють якраз тоді, коли пробіл має бути видно і він не має схлопуватися.

  3. __proto__ і Object.create(null)

  Звичайний {} не порожній: він успадковує від Object.prototype властивості constructor, toString, hasOwnProperty і спеціальний __proto__. Ось що я отримав у Node:

  ```
  const a = {};
  a["__proto__"] = "x";   // не створює ключ, а викликає setter прототипу
  a["class"] = "c";
  JSON.stringify(a)        // {"class":"c"}, атрибут __proto__ мовчки зник

  "constructor" in {}      // true, хоча ми нічого не додавали
  ```

  Для нас це дві конкретні проблеми:
  1. <div __proto__="x">: атрибут просто зникає з результату, без помилки.
  2. : якщо перевіряти дублікат через if (name in attrs) чи attrs[name] !== undefined, парсер вирішить, що constructor уже є (його знайде прототип), і за правилом «перший перемагає» викине справжній атрибут.

  Object.create(null) створює об'єкт без прототипу, тобто справді порожній. Обидва ключі там звичайні:
  ```
  const b = Object.create(null);
  b["__proto__"] = "x"; b["constructor"] = "y";
  JSON.stringify(b)        // {"__proto__":"x","constructor":"y"}
  ```

  Скажу чесно, бо на рев'ю можуть уточнити: у нашому коді глобального prototype pollution не буде. Він виникає, коли код робить щось на кшталт obj[a][b] = v з a = "__proto__", і тоді b з'являється в усіх об'єктах програми. У нас один рівень вкладеності і значення-рядки, тож реальний ризик — це втрачені або неправильно відкинуті атрибути. Map не підходить: JSON.stringify(new Map()) дає {}.

  ---

  🗺️ Роадмеп

| # | Етап | Статус |
|---|---|---|
| 1 | Розбір завдання | ✅ |
| 2 | Git init + перший коміт | ✅ |
| 3 | Список edge cases | ✅ |
| 4 | Формат JSON + рішення | ✅ |
| 5 | README з рішеннями | ⏳ зараз |
| 6 | Семпли в html_samples/ | ⏳ зараз |

| | | |
|---|---|---|
| 7 | Токенізатор: теги, атрибути, текст | |
| 8 | Токенізатор: коментарі, doctype, raw text, EOF | |
| 9 | Декодування entities | |
| 10 | Tree builder: стек, void, неявне закриття, svg, warnings | |
| 11 | Тест-скрипт на всіх семплах + виправлення багів | |
| 12 | Перевірка в index.html у браузері | |
| 13 | Чистка коду, фінальний README | |
| 14 | ai_help/: експорт цієї розмови | |
| 15 | Git (нагадаю) + фінальний чекліст | |

Етапи 7–11 займають основний обсяг роботи, тому відсотки рахую з урахуванням ваги етапів, а не просто за кількістю.

Тепер пишу README. Пишу англійською, як і оригінал завдання. Оригінальний текст лишається в першому коміті.

● Write(README.md)
  ⎿ Added 99 lines, removed 29 lines
```
      1 -# Jito's Software Development Intern "html2json" Test Task
      1 +# html2json
      2
      3 -## Task Rationale
      4 -This task is designed to evaluate how well you solve problems
        -without having every detail explicitly provided and to assess
        -the quality of your deliverables. This type of task isn't nece
        -ssarily reflective of your future work but aims to help us und
        -erstand your thought process and reasoning in the context of s
        -oftware development.
      3 +A hand-written HTML parser that converts an HTML string into a
        + JSON tree.
      4 +No DOM parser (`DOMParser`, `innerHTML`, etc.) is used.
      5
      6 -## Assignment
      7 -Your task is to implement a function called `html2json`, which
        - converts HTML data into a JSON representation.
      8 -AI tools usage is <b>REQUIRED</b>. Is is required that you pro
        -vide your entire conversation history by attaching a link to t
        -he dialogue. Therefore, keep all your research within a single
        - conversation and submit the link along with your task.
      6 +## Usage
      7
     10 -## Expected repository structure
     11 -- `html2json.js` - This file should contain your implementatio
        -n of the html2json function.
     12 -- `html_samples/` folder - Include files with a text that you
        -used as samples to test your function.
     13 -- `index.html` - The initial file we provided. You can leave i
        -t unchanged, but please include it in the archive.
     14 -- `ai_help/` folder - If you used any resources for code gener
        -ation:
     15 -- Create a file named `chatgpt_chat.txt` with a link to the Ch
        -atGPT chat used.
     16 -- For any other AI resources, attach relevant `.pdf`, `.png`,
        -or `.mp4` files showing how you used them.
     17 -- You can optionally update `README.md` completely if you want
        - to add explanations of your reasoning or any other comments.
      8 +Open `index.html` in a browser, paste HTML into the left field
        + and press **Convert to JSON**.
      9
     19 -## Key Points for Evaluation
     20 -- Coverage of various HTML structures and different sizes.
     21 -- The code <b>MUST NOT</b> crash.
     22 -- Code cleanliness and formatting.
     23 -- Using a DOM parser is not allowed.
     24 -- How effectively you handled unexpected scenarios, such as si
        -tuations where your code received valid HTML but still crashed
```

```
- or produced incorrect results. We will evaluate your ability
-to anticipate edge cases and ensure robustness in your solutio
-n.
+```js
+html2json('<p class="a">Hi</p>');
+```

-## P.S. from the team
-Please focus on quality rather than speed. Quality in this con
-text means ensuring your solution is well thought-out, robust,
- and free of obvious issues. The speed of delivery will <b>NOT
-</b> be prioritized, so take the necessary time to research an
-d refine your approach, as long as you complete the task withi
-n the specified timeframe.
-Before submitting your final results, double or even triple-ch
-eck everything:
-- Verify that all links you provide are accessible in incognit
-o mode, as broken links will result in your submission <b>NOT<
-/b> being reviewed.
-- Just before submitting, test your code again to ensure it st
-ill functions correctly and handles the html samples without c
-rashing. If your code crashes or fails on your own samples, it
- will be treated as a failed submission.
-- Make sure all items are included according to the [Expected
-Deliverables](#expected-deliverables) section. If any required
- files or information are missing, we will <b>NOT</b> be able
-to review your task, and it will be <ins>treated as failed</in
-s>.
-- Jito's senior developer will thoroughly review your solution
-. Based on this review, if deemed appropriate, you may be invi
-ted for a technical code review. This will include questions a
-bout the code, your understanding, and the reasoning behind yo
-ur solution choices.
-- The best indicator that you've done your best is the feeling
- of confidence when submitting, knowing that you have thorough
-ly checked your work and cannot think of anything more to impr
-ove.
-- You can view test task template [here](https://jito-dev.gith
-ub.io/jito-intern-test-task/)
+## Output format
+
+The result is always a single `root` node, because an HTML fra
+gment can have several top-level nodes.
+
+```json
+{
+  "type": "root",
+  "children": [
+    {
+      "type": "element",
+      "tag": "p",
+      "attributes": { "class": "a" },
+      "children": [{ "type": "text", "content": "Hi" }]
+    }
+  ],
+  "warnings": []
+}
+```
+
+| Node type | Fields |
+|---|---|
+| `root` | `children`, `warnings` |
+| `element` | `tag`, `attributes`, `children` |
+| `text` | `content` |
+| `comment` | `content` |
+| `doctype` | `content` (e.g. `"html"`) |
+
+- `attributes` and `children` are **always present** on elemen
+ts (also on void elements like `<br>`), so consumers never hav
+e to check whether a field exists.
+- Nodes have **no `parent` field**: it would create circular r
+eferences and `JSON.stringify` would throw.
+- `warnings` is a list of `{ message, line, column }` objects
+describing problems found in the input (unclosed tags, stray e
+nd tags, etc.). Parsing never fails because of them.
+
```

```
45 +## Design decisions
46 +
47 +### General
48 +- **The input is returned as written.** `<html>`, `<head>` and
   + `<body>` are not inserted automatically, unlike in a browser.
49 +- **Non-string input** (`null`, `undefined`, numbers, ...) ret
   +urns an empty root instead of throwing.
50 +
51 +### Tags
52 +- Tag and attribute names are **lowercased**, because HTML is
   +case-insensitive. **Exception:** inside `<svg>` and `<math>` t
   +he case is kept, since these languages are case-sensitive (`vi
   +ewBox`).
53 +- **Void elements** (`area, base, br, col, embed, hr, img, inp
   +ut, link, meta, source, track, wbr`) never have children and n
   +eed no end tag.
54 +- **`<tag/>` is treated as closed for any tag**, including `<d
   +iv/>`. A browser ignores the slash on non-void HTML elements;
   +we follow the author's obvious intent instead (a habit from XM
   +L/JSX).
55 +
56 +### Attributes
57 +- An attribute without a value (`disabled`) gets an empty stri
   +ng `""`, which is what browsers do.
58 +- On **duplicate attributes the first one wins**, which is wha
   +t browsers do.
59 +- Attributes are stored in an object created with `Object.crea
   +te(null)`. With a plain `{}`, an attribute named `__proto__` w
   +ould silently disappear, and `constructor` would look like an
   +existing duplicate because of the inherited prototype.
60 +
61 +### Text
62 +- Text content is **not trimmed**: leading and trailing spaces
   + can be meaningful (`Hello <b>world</b>`).
63 +- **Whitespace-only text that contains a line break is dropped
   +**, because that is indentation from formatting the source. A
   +whitespace-only text without a line break (`<b>a</b> <i>b</i>`
   +) is kept, because it is a real space between words. Inside `<
   +pre>` and `<textarea>` all whitespace is kept.
64 +- "Whitespace" means the HTML set `[ \t\n\r\f]`, not JS `\s`.
   +`\s` also matches ` ` (U+00A0), and that is content.
65 +- Adjacent text nodes are merged into one.
66 +
67 +### Character references (entities)
68 +- Numeric (`&#169;`, `&#xA9;`) and the most common named refer
   +ences (`&amp;`, `&lt;`, `&gt;`, `&quot;`, `&apos;`, ` `,
   +`&copy;`, ...) are decoded in text and attribute values.
69 +- Unknown references (`&foo;`) and a lone `&` are kept as writ
   +ten.
70 +- The full HTML list has over 2000 named references, so only a
   + subset is supported. Adding more only requires extending the
   +table.
71 +
72 +### Raw text elements
73 +- The content of `<script>` and `<style>` is kept **exactly as
   + written**, as one text node: no tags, no entity decoding.
74 +- The content of `<textarea>` and `<title>` contains no tags e
   +ither, but entities **are** decoded (the same as in browsers).
75 +
76 +### Comments and other markup
77 +- `<!-- ... -->` becomes a `comment` node. An unclosed comment
   + runs to the end of the input.
78 +- `<!DOCTYPE ...>` becomes a `doctype` node (case-insensitive)
   +.
79 +- `<![CDATA[...]]>` and `<?...?>` are treated as comments, as
   +browsers do in HTML.
80 +- A `<` that does not start a tag (`a < b`, `<3`) is plain tex
   +t.
81 +
82 +### Invalid HTML
83 +- **Implicit closing** for the most common cases:
84 +  - `<p>` is closed by another `<p>` or by a block-level start
   + tag (`div`, `ul`, `table`, `h1`–`h6`, ...).
85 +  - `<li>`, `<option>`, `<tr>`, `<td>`/`<th>` are closed by a
   +sibling of the same kind, but **only if the open element is th
   +e current node** (top of the stack). Because of this, a nested
```

```
   + `<ul>` never closes the `<li>` of an outer list.
86 +- **An end tag that is not open** (`</span>` without `<span>`)
   + is ignored, with a warning.
87 +- **An end tag that is open but not current** closes all eleme
   +nts above it: in `<b><i>x</b></i>`, `</b>` closes both element
   +s and `</i>` is ignored.
88 +- **Tags still open at the end of the input** are closed, with
   + a warning.
89 +- **Input that ends inside a tag** (`<div class="a`) is not lo
   +st: the content read so far is kept, with a warning.
90 +
91 +### Robustness
92 +- The tree is built iteratively with an explicit stack, withou
   +t recursion, so deeply nested input cannot overflow the call s
   +tack.
93 +- The input is scanned in a single pass, with no regular expre
   +ssions that could backtrack catastrophically.
94 +
95 +## Known limitations
96 +- Whitespace that contains a line break between inline element
   +s (`<span>a</span>\n<span>b</span>`) is dropped, so the text r
   +eads `ab` while a browser shows `a b`. Telling inline from blo
   +ck elements depends on CSS, which cannot be known from HTML al
   +one.
97 +- Only the most common implicit-closing rules are implemented,
   + not the full HTML5 tree construction algorithm (no "adoption
   +agency" for misnested formatting tags, no table foster parenti
   +ng).
98 +- Only a subset of named character references is supported.
99 +
100 +## Repository structure
101 +- `html2json.js`: the implementation.
102 +- `index.html`: the provided page (unchanged).
103 +- `html_samples/`: HTML files used to test the function.
104 +- `ai_help/`: the AI conversation used while working on the ta
    +sk.
```

- README готовий. Тепер семпли, по одному файлу на кожну категорію edge cases:

  Ran 1 shell command (ctrl+o to expand)

- Записав рішення в README і створив 23 семпли в html_samples/. Ти перейшов на етап 7 з 15, приблизно 30%. Етапи 1–6 готові, але основна робота (код) ще попереду.

  README

  Я переписав README.md під наше рішення: формат JSON, усі рішення з обґрунтуванням і розділ «Known limitations», куди потрапив твій кейс a\nb. Оригінальний текст завдання лишився в першому коміті.

  Три речі я додав сам, подивись, чи згоден:
  - Формат warnings: { message, line, column }. Номер рядка й колонки дуже допомагає при дебазі.
  - EOF посеред тегу (<div class="a): лишаю те, що вже встигли прочитати, і додаю warning. Браузер у такому випадку просто викидає тег, але я вважаю, що втрачати дані гірше.
  - У розділі «Robustness» є дві обіцянки: дерево будується без рекурсії, і немає regex, які можуть зависнути на великому вході. Їх треба буде дотримати в коді.

  Семпли

| Файли | Що перевіряють |
|---|---|
| 01–02 | Приклади 1 і 2 з index.html |
| 03 | void-теги і , , <br></br> |
| 04 | Усі види атрибутів, дублікати, регістр, __proto__/constructor, <div a="1"b="2">, <div =broken> |
| 05 | Entities: відомі, числові, без ;, невідомі, самотній &, a < b |

| | |
|---|---|
| 06 | Пробіли: trim, a b,  , <pre>, <textarea> |
| 07 | script/style/textarea/title, </SCRIPT > |
| 08 | Коментарі, doctype, CDATA, <?xml?> |
| 09 | Неявне закриття p, li (зокрема вкладений список), option, tr/td/th |
| 10 | Незакриті теги, зайві закриваючі, неправильна вкладеність |
| 11–15 | Обрив файлу посеред атрибута, коментаря, скрипту, після < і </ |
| 16 | SVG і MathML (регістр viewBox) |
| 17 | Кирилиця, емодзі, RTL-письмо |
| 18–21 | Порожній файл, файл із самих пробілів, текст без тегів, BOM + \r\n |
| 22 | 10 000 вкладених  (перевірка stack overflow) |
| 23 | Великий документ на 1,5 МБ, 10 000 секцій (перевірка швидкодії) |

  Семпли 22 і 23 я згенерував скриптом. Файли 11–15 навмисно без переносу рядка
  в кінці, інакше EOF не буде там, де треба. Порожній .gitkeep з html_samples/
  видалив, бо папка вже не порожня.

  Правильний результат для кожного семплу ми ще не перевіряли, це буде на етапі
  11. Зараз це лише вхідні дані.

  Наступний етап 7: токенізатор (теги, атрибути, текст). Коли почнемо, я спершу
  поясню, на які стани поділити state machine, а потім пишемо код.

✻ Baked for 1m 59s · done Saturday 10:45 PM

※ recap: Робимо тестове завдання від Jito: свій парсер html2json без DOMParser. Рішення вже записані в README, 23 семпли готові, ми на етапі 7 з 15 (приблизно 30%). Далі розберемо стани токенізатора і почнемо писати код. (disable recaps in /config)

❯ по трьох твоїх доповненнях згоден: warnings з line/column, обрив тегу лишаємо
  з warning, і без рекурсії та regex.
  ще глянув семпли, ок. давай етап 7, токенізатор. поясни стани state machine,
  і я хочу щоб: був один прохід по рядку з індексом, без slice на кожному
  символі, < вважався тегом тільки якщо далі буква, / , ! або ?, токенізатор
  сам переключався в raw text після script/style/textarea/title

  і два питання:
  1. як буде рахуватись line і column? \r\n це один перенос чи два?
  2. як шукаємо кінець script, якщо в js буде рядок "</script>"?

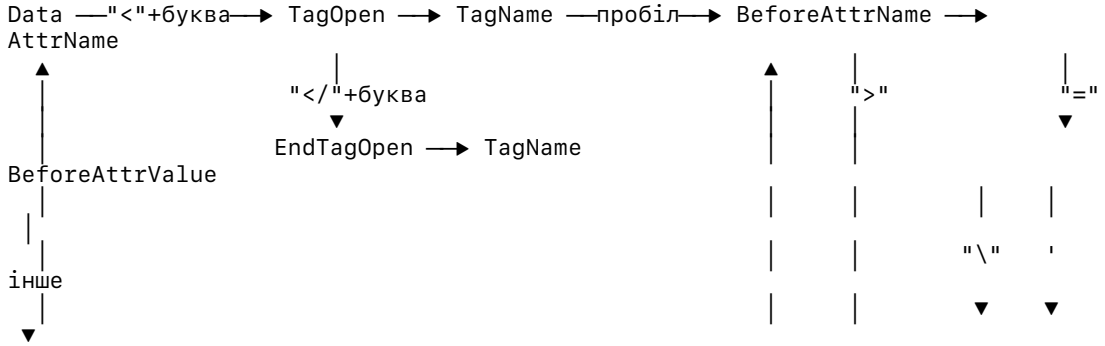
  пиши по частинах і після кожної коротко поясни, шо робить код

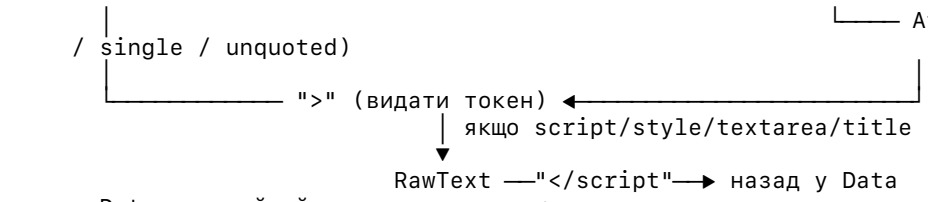
  Read 1 file, ran 1 shell command (ctrl+o to expand)

● Перед кодом розберемо стани і твої два питання.

  Стани state machine

  У специфікації HTML у токенізатора близько 80 станів. Для нашого обсягу
  потрібні ось ці:






- Data: звичайний текст, шукаємо <.
- TagOpen / EndTagOpen: побачили < або </ і вирішуємо, чи це справді тег.
- TagName: читаємо ім'я до пробілу, / або >.
- BeforeAttrName / AttrName / BeforeAttrValue / AttrValue: атрибути.
- SelfClosing: / перед >.
- RawText: вміст script/style/textarea/title.
- Коментарі й doctype (MarkupDeclaration, Comment, Doctype, BogusComment) будуть на етапі 8.

Як це реалізувати. Можна зробити одну змінну state і величезний switch на кожен символ, як у специфікації. Але такий код важко читати. Я пропоную функцію на кожну групу станів: readTag, readAttribute, findRawTextEnd. Стан тоді визначається тим, у якій функції ми зараз. Це той самий state machine, але простіший для читання. На рев'ю можна так і пояснити: «стани з специфікації згруповані у функції, перехід між станами — це виклик функції або вихід з циклу».

Твої вимоги в цьому дизайні:
- Один прохід з індексом. Є один pos, і він тільки росте. slice викликається раз на токен (ім'я, значення, шматок тексту), а не на кожен символ. Пошук швидкий завдяки indexOf: це пошук нативним кодом, і він теж іде тільки вперед.
- < починає тег тільки перед буквою, /, !, ?. На цьому етапі реалізую букву і /+буква. !, ? і випадки на кшталт </> або </ 3 додамо на етапі 8 разом із коментарями. Інакше зараз їх не буде чим обробити.
- Raw text токенізатор вмикає сам, одразу після start-тегу.

1. line і column

- \r\n вважаємо одним переносом. Одиночний \r теж є переносом. Специфікація HTML прямо каже на першому кроці замінити \r\n і \r на \n (input preprocessing), і браузер робить так само. Тому перед токенізацією нормалізуємо вхід, тоді й рахувати просто. Побічний ефект: у <pre>/<textarea> \r зникне, як і в браузері. BOM на початку теж прибираємо.
- line і column рахуються з 1, як в усіх редакторах.
- column рахуємо в UTF-16 code units, тобто як індекс у JS-рядку. Емодзі займе 2 колонки. Це трейдоф, який варто записати в README.
- Як рахуємо. Токенізатор не відстежує рядки на кожному символі, а просто записує в токен start, тобто зсув від початку рядка. Перетворювати offset → line:column будемо лише тоді, коли створюємо warning: один раз будуємо масив початків рядків, а далі шукаємо бінарним пошуком. Звичайний прохід так не сповільнюється. Це етап 10.

2. "</script>" у JS

Це питання з підступом: ми не шукаємо JS-рядки, і браузер теж не шукає. Raw text закінчується на першому </script, після якого йде пробіл, /, > або кінець файлу. Тобто

<script>var s = "</script>";</script>
ламає і браузер: скрипт закінчиться на var s = ", а "; стане звичайним текстом. Тому в JS і пишуть "<\/script>". Ми повторюємо поведінку браузера. Розбирати JS, щоб розуміти, де в ньому рядки, — завдання JS-парсера, а не HTML-парсера.

Два нюанси:
- </scripts> або </scriptx не закривають скрипт, бо після імені має бути роздільник.
- <script /> за нашим рішенням про  вважаємо закритим, тому в raw text не переходимо.

Файл html2json.js має переноси рядків CRLF. В кінці я приведу всі переноси до CRLF, щоб не змішувати стилі.

---

Частина 1: допоміжні функції і головний цикл

● Update(html2json.js)

Added 70 lines

```
  };
 }

+/* ---------- Tokenizer ---------- */
+
+// Content is read as-is until the matching end tag: no tags inside.
+const RAW_TEXT_TAGS = new Set(["script", "style", "textarea", "title"]);
+
+function isAsciiWhitespace(ch) {
+  return ch === " " || ch === "\t" || ch === "\n" || ch === "\r" || ch === "\f";
+}
+
+function isAsciiLetter(ch) {
+  return (ch >= "a" && ch <= "z") || (ch >= "A" && ch <= "Z");
+}
+
+function skipWhitespace(input, pos) {
+  while (pos < input.length && isAsciiWhitespace(input[pos])) pos++;
+  return pos;
+}
+
+// "<" starts a tag only before a letter ("<div") or "/" + letter ("</div").
+// Anything else ("a < b", "<3") is plain text.
+function isTagStart(input, pos) {
+  const next = input[pos + 1];
+  if (isAsciiLetter(next)) return true;
+  return next === "/" && isAsciiLetter(input[pos + 2]);
+}
+
+/*
+  Splits the input into a flat list of tokens in a single forward pass:
+    { type: "startTag", name, attributes, selfClosing, terminated, start, end }
+    { type: "endTag",   name, attributes, selfClosing, terminated, start, end }
+    { type: "text",     content, start, end }
+  `start` and `end` are offsets in the input, used later for warnings.
+*/
+function tokenize(input) {
+  const tokens = [];
+  let pos = 0;
+  let textStart = 0;
+
+  function pushText(end) {
+    if (end > textStart) {
+      tokens.push({ type: "text", content: input.slice(textStart, end), start: textStart, end });
+    }
+  }
+
+  while (pos < input.length) {
+    const lt = input.indexOf("<", pos);
+    if (lt === -1) break;
+    if (!isTagStart(input, lt)) {
+      pos = lt + 1;
+      continue;
+    }
+
+    pushText(lt);
+    const tag = readTag(input, lt);
+    tokens.push(tag);
+    pos = tag.end;
+
+    const name = tag.name.toLowerCase();
+    if (tag.type === "startTag" && !tag.selfClosing && RAW_TEXT_TAGS.has(name)) {
+      textStart = pos;
+      pos = findRawTextEnd(input, pos, name);
```

```
80 +      pushText(pos);
81 +    }
82 +    textStart = pos;
83 +  }
84 +
85 +  pushText(input.length);
86 +  return tokens;
87 +}
88 +
89  function showExample1() {
90    const htmlExample = `<!DOCTYPE html>
91  <html lang="en">
```

● Update(html2json.js)
  ⎿ Added 1 line, removed 1 line

```
18
19  /* ---------- Tokenizer ---------- */
20
21 -// Content is read as-is until the matching end tag: no tags in
   -side.
21 +// No tags inside: everything up to the matching end tag is tex
   +t.
22  const RAW_TEXT_TAGS = new Set(["script", "style", "textarea", "
    title"]);
23
24  function isAsciiWhitespace(ch) {
```

● Що робить частина 1:
- isAsciiWhitespace перевіряє пробіл за HTML-визначенням ([ \t\n\r\f]), без \s. \r у списку лишається для надійності, хоча після нормалізації його вже не буде.
- isTagStart реалізує твоє правило: < вважається тегом лише перед буквою або перед /+буква. Якщо ch вийшов за межі рядка, input[pos] дає undefined, і порівняння просто повертає false, без падіння.
- tokenize — це стан Data. Через indexOf стрибаємо до наступного <. Якщо це не тег, < лишається частиною тексту, і ми йдемо далі. Якщо тег, спершу «виштовхуємо» накопичений текст (pushText), а потім читаємо тег.
- Текст не склеюється по символу. Ми пам'ятаємо лише textStart і в потрібний момент робимо один slice.
- Після start-тегу script/style/textarea/title переходимо в RawText: весь вміст до закриваючого тегу стає одним text-токеном. Сам </script> у наступній ітерації прочитається як звичайний end-тег.
- Імена тут не приводяться до нижнього регістру. Токенізатор не знає, чи ми всередині <svg>, тому регістром займеться tree builder. toLowerCase() тут лише для перевірки на raw text.
- textarea і title поки в одному наборі з script/style. На етапі 9 розділимо їх, бо в перших двох entities декодуються.

Частина 2: читання тегу

● Update(html2json.js)
  ⎿ Added 51 lines

```
86    return tokens;
87  }
88
89 +// Reads "<name ...>" or "</name ...>" starting at the "<".
90 +// If the input ends inside the tag, the tag is kept with term
   +inated: false.
91 +function readTag(input, start) {
92 +  const isEndTag = input[start + 1] === "/";
93 +  let pos = start + (isEndTag ? 2 : 1);
94 +
95 +  const nameStart = pos;
96 +  while (pos < input.length && !isTagNameEnd(input[pos])) pos+
   ++;
97 +  const name = input.slice(nameStart, pos);
98 +
99 +  const attributes = [];
100 +  let selfClosing = false;
101 +  let terminated = false;
102 +
103 +  while (pos < input.length) {
104 +    const ch = input[pos];
105 +    if (isAsciiWhitespace(ch)) {
106 +      pos++;
107 +    } else if (ch === ">") {
```

```
108 +      pos++;
109 +      terminated = true;
110 +      break;
111 +    } else if (ch === "/") {
112 +      pos++;
113 +      if (input[pos] === ">") {
114 +        pos++;
115 +        selfClosing = true;
116 +        terminated = true;
117 +        break;
118 +      }
119 +      // A "/" not followed by ">" is ignored, as in browsers.
120 +    } else {
121 +      pos = readAttribute(input, pos, attributes);
122 +    }
123 +  }
124 +
125 +  return {
126 +    type: isEndTag ? "endTag" : "startTag",
127 +    name,
128 +    attributes,
129 +    selfClosing,
130 +    terminated,
131 +    start,
132 +    end: pos,
133 +  };
134 +}
135 +
136 +function isTagNameEnd(ch) {
137 +  return isAsciiWhitespace(ch) || ch === "/" || ch === ">";
138 +}
139 +
140  function showExample1() {
141    const htmlExample = `<!DOCTYPE html>
142  <html lang="en">
```

- Ім'я тегу читається до пробілу, / або >, а цикл атрибутів пропускає зайві символи, включно з end-тегами, і при EOF повертає прочитане з terminated: false. Далі переходжу до атрибутів (частина 3).

- Update(html2json.js)
  └ Added 39 lines

```
137    return isAsciiWhitespace(ch) || ch === "/" || ch === ">";
138  }
139
140 +// Reads one attribute (name, optional "=" and value), adds it
    + to `attributes`
141 +// and returns the position right after it.
142 +function readAttribute(input, pos, attributes) {
143 +  const nameStart = pos;
144 +  // The first character always belongs to the name, even "="
    +(<div =a>).
145 +  pos++;
146 +  while (pos < input.length && !isAttributeNameEnd(input[pos])
    +) pos++;
147 +  const name = input.slice(nameStart, pos);
148 +
149 +  pos = skipWhitespace(input, pos);
150 +  if (input[pos] !== "=") {
151 +    attributes.push({ name, value: "" });
152 +    return pos;
153 +  }
154 +
155 +  pos = skipWhitespace(input, pos + 1);
156 +  const quote = input[pos];
157 +  let value;
158 +
159 +  if (quote === '"' || quote === "'") {
160 +    const close = input.indexOf(quote, pos + 1);
161 +    const valueEnd = close === -1 ? input.length : close;
162 +    value = input.slice(pos + 1, valueEnd);
163 +    pos = close === -1 ? valueEnd : close + 1;
164 +  } else {
165 +    // Unquoted: ends at whitespace or ">". A "/" is part of t
    +he value (href=/a/).
166 +    const valueStart = pos;
```

```
167 +    while (pos < input.length && !isAsciiWhitespace(input[pos]
    +) && input[pos] !== ">") pos++;
168 +    value = input.slice(valueStart, pos);
169 +  }
170 +
171 +  attributes.push({ name, value });
172 +  return pos;
173 +}
174 +
175 +function isAttributeNameEnd(ch) {
176 +  return isAsciiWhitespace(ch) || ch === "/" || ch === ">" ||
    +ch === "=";
177 +}
178 +
179  function showExample1() {
180    const htmlExample = `<!DOCTYPE html>
181  <html lang="en">
```

- Частина 3 описує, як токенізатор читає імена та значення атрибутів: імена йдуть до пробілу, /, > або =, лапки закривають значення, а дублікати й регістр лишаються як є, щоб їх обробляв tree builder. Далі йде частина 4 про кінець raw text.

- Update(html2json.js)
  ⎿ Added 22 lines

```
176    return isAsciiWhitespace(ch) || ch === "/" || ch === ">" ||
     ch === "=";
177  }
178
179 +/*
180 +  Finds where raw text of `tagName` ends: the first "</tagName
    +" (case-insensitive)
181 +  followed by whitespace, "/", ">" or the end of input. Like b
    +rowsers, it does not
182 +  look inside JS strings, so "</script>" in a script string al
    +so ends the script.
183 +  Returns the input length if there is no end tag.
184 +*/
185 +function findRawTextEnd(input, pos, tagName) {
186 +  let candidate = input.indexOf("</", pos);
187 +  while (candidate !== -1) {
188 +    const nameEnd = candidate + 2 + tagName.length;
189 +    const after = input[nameEnd];
190 +    const nameMatches = input.slice(candidate + 2, nameEnd).to
    +LowerCase() === tagName;
191 +    if (nameMatches && (after === undefined || isTagNameEnd(af
    +ter))) return candidate;
192 +    candidate = input.indexOf("</", candidate + 2);
193 +  }
194 +  return input.length;
195 +}
196 +
197 +if (typeof module !== "undefined") {
198 +  module.exports = { html2json, tokenize };
199 +}
200 +
201  function showExample1() {
202    const htmlExample = `<!DOCTYPE html>
203  <html lang="en">
```

- module.exports краще поставити в самий кінець файлу, перенесу:

- Update(html2json.js)
  ⎿ Removed 4 lines

```
194    return input.length;
195  }
196
197 -if (typeof module !== "undefined") {
198 -  module.exports = { html2json, tokenize };
199 -}
200 -
197  function showExample1() {
198    const htmlExample = `<!DOCTYPE html>
199  <html lang="en">
```

  Ran 1 shell command (ctrl+o to expand)

- Тут пояснюється, як парсер визначає кінець RawText: це перше </ім'я без урахування регістру, після якого йде пробіл, /, > або кінець файлу. Пошук кандидатів іде через indexOf("</") лінійно, а якщо закриваючого тегу немає, решта файлу стає текстом із попередженням. Далі буде перевірка.

  Ran 1 shell command (ctrl+o to expand)
  └ Found 3 new diagnostic issues in 1 file (ctrl+o to expand)

- Токенізатор працює правильно на всіх кейсах з частин 1–4. Додам у README рішення, які ми прийняли на цьому етапі:

  Ran 1 shell command (ctrl+o to expand)

- Токенізатор готовий і працює на всіх тестових кейсах і семплах. Етап 7 з 15 завершено, приблизно 40%.

  Що перевірив

  Запустив його на 13 точкових кейсах і на всіх 23 семплах. Нічого не впало і не зависло. Ключові результати:

| Вхід | Результат |
|---|---|
| title="a > b", 'he said "hi"' | > і лапки всередині значення читаються правильно |
| a="1"b="2", =broken, class = "a" | a, b, =broken, class — як у браузері |
| <a href=/path/> | href="/path/", не self-closing |
| <div / class="a"> | одиночний / проігноровано |
| a < b, 1<2, <3 | один text-токен, тегів немає |
| <script>... " " ...</SCRIPT > |  лишився текстом, </SCRIPT > закрив скрипт |
| <script>var s = "</script>";</script> | скрипт закрився на першому </script>, далі текст "; і зайвий </script>. Так само робить браузер |
| </styles> у <style> | не закриває style |
| <div class="a (обрив) | токен збережено з UNTERMINATED, щоб потім дати warning |
| </div title=">"> | > у лапках не обриває end-тег |

  Швидкодія: 10 000 вкладених  займають 4 мс, файл на 1,5 МБ (170 000 токенів) — 19 мс. Тобто час лінійний.

  Що ще змінилось

  - README: додав наші рішення про line/column (\r\n вважається одним переносом, column в UTF-16), нормалізацію BOM і \r, "</script>" у JS і <script />. У «Known limitations» з'явився новий пункт: <style>/<script>/<title> всередині <svg> теж читаються як raw text, хоча в браузері там це звичайні SVG-елементи. Причина та сама: токенізатор не знає контексту дерева.
  - html2json.js: html2json поки повертає заглушку. Підключимо до неї парсер, коли буде tree builder (етап 10).
  - IDE-підказки «convertHtml2JsonAndSet/showExample1/showExample2 declared but never read» хибні. Ці функції викликаються з onclick в index.html, а TypeScript цього не бачить. Можна ігнорувати.

  На рев'ю, мабуть, спитають, чому атрибути в токені лежать масивом, а не об'єктом. Відповідь: токенізатор описує вхід як є, з дублікатами й у правильному порядку. Що з цього потрапить у результат (перший атрибут, нижній регістр), вирішує tree builder.

  Наступний етап 8: коментарі, <!DOCTYPE>, <![CDATA[, <?...?>, а також </> і </3. Тоді < почне вважатися тегом і перед ! та ?, як ти хотів.

✻ Churned for 3m 34s · done Saturday 11:11 PM

※ recap: Ти пишеш свій HTML-парсер `html2json` для тестового Jito, і токенізатор (етап 7 з 15, приблизно 40%) уже готовий і перевірений на семплах. Наступний крок — етап 8: коментарі, doctype, CDATA і `<?...?>`. (disable recaps in /config)

❯ перевірив таблицю, виглядає ок. парочку  питаннь перед етапом 8:

  1. якщо лапка незакрита, типу <div class="a>текст , то valueEnd =
  input.length і весь файл до кінця стане значенням атрибута? це норм чи треба
  шось робити? хоча б warning
  2. ти казав шо \r\n і BOM нормалізуємо перед токенізацією, але в tokenize я
  цього не бачу. де це буде?
  3. як ти потім знайдеш line і column по start? поясни простими словами шо
  таке бінарний пошук

  і поясни коротко чому indexOf швидший ніж просто цикл по символах
  потім зроби коміт (токенізатор готовий, хочу сейв) і давай етап 8

● Спочатку зроблю коміт, щоб зберегти поточний стан, і заміряю indexOf проти
  циклу, щоб відповідати цифрами, а не на око.

  Ran 2 shell commands (ctrl+o to expand)

● Коміт fa3a04f зроблено. Тепер відповіді на питання.

  1. Незакрита лапка

  Значення атрибута йде не обов'язково до кінця файлу, а до наступної такої
  самої лапки. У <div class="a>текст значенням стане все до наступного " у
  файлі, де б воно не було. Так само поводиться браузер: він теж не вгадує, де
  автор «мав на увазі» кінець. Якщо лапок далі немає, у браузері тег зникає
  зовсім, а в нас він лишається з terminated: false.

  Пробувати «відновитися» евристикою (наприклад, обрізати значення на > або на
  переносі рядка) не варто: тоді на валідному title="a > b" результат був би
  неправильним. Тож поведінку лишаємо, але ти маєш рацію щодо warning. Причому
  warning має бути конкретний: «незакрита лапка в атрибуті class» з позицією
  лапки, а не загальне «щось не так з тегом». Тому на етапі 8 я зроблю так, щоб
  токенізатор теж писав warnings у спільний список { message, offset }. Так буде
  з усіма проблемами, які бачить тільки він: незакрита лапка, незакритий
  коментар, </>. Наприкінці цей список сортуємо за offset, щоб warnings ішли в
  порядку тексту.

  2. Де нормалізація

  Навмисно не в tokenize, а в html2json: html2json спершу готує вхід (перевірка
  типу, BOM, \r\n), а tokenize отримує вже чистий рядок. Так кожна функція
  робить одну річ, а tokenize простіше тестувати окремо. Саму функцію
  normalizeInput додам зараз, а викликати її html2json буде з етапу 10, коли
  з'явиться tree builder.

  3. line/column і бінарний пошук

  Один раз проходимо по тексту і записуємо, з якого зсуву починається кожен
  рядок:
  "ab\ncde\nf"   →   lineStarts = [0, 3, 7]
                              рядок 1  2  3
  Для offset = 5 шукаємо останній початок рядка, який ≤ 5. Це 3, тобто рядок 2,
  а колонка = 5 − 3 + 1 = 3 (символ e).

  Шукати можна було б перебором зліва направо, але масив уже відсортований, тому
  підходить бінарний пошук, як пошук слова в паперовому словнику. Відкриваєш
  посередині: потрібне слово раніше чи пізніше? Половину словника відкидаєш і
  повторюєш. Кожен крок ділить діапазон навпіл, тож на мільйон рядків вистачає
  ~20 кроків замість мільйона.

  Масив будуємо лише тоді, коли є хоч один warning. На чистому HTML це взагалі
  нічого не коштує.

  Чому indexOf швидший

  Заміряв на рядку з 20 млн символів:

| Спосіб | Час |
|---|---|

| | |
|---|---|
| цикл s[i] === "<" | 28 мс |
| цикл s.charCodeAt(i) === 60 | 23 мс |
| s.indexOf("<") | 0,3 мс |

Складність однакова, O(n), бо обидва переглядають кожен символ. Різниця в ціні одного кроку:
- Цикл у JS на кожному символі робить кілька операцій: перевіряє межі, дістає символ (s[i] ще й створює односимвольний рядок), порівнює, збільшує i.
- indexOf реалізований у рушії на C++. Для пошуку одного символу він використовує оптимізовані процедури на кшталт memchr, які за допомогою SIMD-інструкцій процесора перевіряють 16–32 символи за раз.

Головне, що indexOf у нас завжди шукає вперед від поточної позиції і ніколи не повертається назад. Тому весь прохід лишається лінійним. Нижче в коді коментарів це знадобиться: у них я свідомо не шукаю два різні маркери окремими indexOf. Якщо другого маркера у файлі немає, кожен пошук пробігав би до кінця файлу, і на 10 000 коментарів вийшло б $O(n^2)$.

---

Етап 8: коментарі, doctype і решта <!/<?/</

Що каже специфікація і що ми робимо:

| Вхід | Результат |
|---|---|
| <!-- ... --> | comment. Закривається також --!>, як у браузері |
| <!-->, <!---> | порожній коментар (так у специфікації) |
| <!-- без кінця | коментар до кінця файлу + warning |
| <!DOCTYPE ...> (будь-який регістр) | doctype, content — текст після слова DOCTYPE без пробілів по краях |
| <![CDATA[...]]>, <!щось> | bogus comment до першого > + warning |
| <?xml ...?> | bogus comment, вміст ?xml ...? + warning |
| </ 3>, </#> | bogus comment + warning |
| </> | повністю ігнорується + warning |
| </ в кінці файлу | текст </ |

Bogus comment означає, що браузер не розуміє цю розмітку, але й не показує її як текст, а мовчки перетворює на коментар. Ми робимо так само, просто додаємо warning.

Read 1 file (ctrl+o to expand)

● Частина 1: нормалізація входу і вибір, що стоїть після <

● Update(html2json.js)
  ⎿ Added 55 lines, removed 18 lines
```
    34    return pos;
    35  }
    36
    37 -// "<" starts a tag only before a letter ("<div") or "/" + lett
       -er ("</div").
    38 -// Anything else ("a < b", "<3") is plain text.
    39 -function isTagStart(input, pos) {
    40 -  const next = input[pos + 1];
    41 -  if (isAsciiLetter(next)) return true;
    42 -  return next === "/" && isAsciiLetter(input[pos + 2]);
    37 +function trimAsciiWhitespace(text) {
    38 +  let start = 0;
    39 +  let end = text.length;
    40 +  while (start < end && isAsciiWhitespace(text[start])) start++
       +;
```

```
 41 +  while (end > start && isAsciiWhitespace(text[end - 1])) end--
    +;
 42 +  return text.slice(start, end);
 43  }
 44
 45 +// Input preprocessing from the HTML spec: drop a leading BOM,
 46 +// turn "\r\n" and a lone "\r" into "\n".
 47 +function normalizeInput(input) {
 48 +  const withoutBom = input.charCodeAt(0) === 0xfeff ? input.sli
    +ce(1) : input;
 49 +  return withoutBom.replace(/\r\n?/g, "\n");
 50 +}
 51 +
 52  /*
 53    Splits the input into a flat list of tokens in a single forwa
     rd pass:
 54      { type: "startTag", name, attributes, selfClosing, terminat
     ed, start, end }
 55      { type: "endTag",   name, attributes, selfClosing, terminat
     ed, start, end }
 56      { type: "text",     content, start, end }
 50 -  `start` and `end` are offsets in the input, used later for wa
    -rnings.
 57 +    { type: "comment",  content, start, end }
 58 +    { type: "doctype",  content, start, end }
 59 +  `start` and `end` are offsets in the input. Problems found in
    + the input are
 60 +  added to `warnings` as { message, offset }.
 61  */
 52 -function tokenize(input) {
 62 +function tokenize(input, warnings = []) {
 63    const tokens = [];
 64    let pos = 0;
 65    let textStart = 0;
...
 73    while (pos < input.length) {
 74      const lt = input.indexOf("<", pos);
 75      if (lt === -1) break;
 66 -    if (!isTagStart(input, lt)) {
 76 +
 77 +    const token = readMarkup(input, lt, warnings);
 78 +    if (token === null) {
 79        pos = lt + 1;
 80        continue;
 81      }
 82
 83      pushText(lt);
 72 -    const tag = readTag(input, lt);
 73 -    tokens.push(tag);
 74 -    pos = tag.end;
 84 +    if (token.type !== "ignored") tokens.push(token);
 85 +    pos = token.end;
 86
 76 -    const name = tag.name.toLowerCase();
 77 -    if (tag.type === "startTag" && !tag.selfClosing && RAW_TEXT
    -_TAGS.has(name)) {
 78 -      textStart = pos;
 79 -      pos = findRawTextEnd(input, pos, name);
 80 -      pushText(pos);
 87 +    if (token.type === "startTag" && !token.selfClosing) {
 88 +      const name = token.name.toLowerCase();
 89 +      if (RAW_TEXT_TAGS.has(name)) {
 90 +        textStart = pos;
 91 +        pos = findRawTextEnd(input, pos, name);
 92 +        pushText(pos);
 93 +      }
 94      }
 95      textStart = pos;
 96    }
...
 99    return tokens;
100  }
101
102 +/*
103 +  Decides what starts at the "<" and reads it. Returns null if
    + the "<" is plain
```

```
104 +  text ("a < b", "<3", "</" at the end of input). For "</>", w
    +hich browsers drop,
105 +  returns a token of type "ignored" so the caller can skip it.
106 +*/
107 +function readMarkup(input, pos, warnings) {
108 +  const next = input[pos + 1];
109 +  if (isAsciiLetter(next)) return readTag(input, pos, warnings
    +);
110 +  if (next === "!") return readMarkupDeclaration(input, pos, w
    +arnings);
111 +  if (next === "?") {
112 +    return readBogusComment(input, pos, pos + 1, `"<?" markup
    +is treated as a comment`, warnings);
113 +  }
114 +  if (next !== "/") return null;
115 +
116 +  const afterSlash = input[pos + 2];
117 +  if (isAsciiLetter(afterSlash)) return readTag(input, pos, wa
    +rnings);
118 +  if (afterSlash === ">") {
119 +    warnings.push({ message: `Empty end tag "</>" is ignored`,
    + offset: pos });
120 +    return { type: "ignored", start: pos, end: pos + 3 };
121 +  }
122 +  if (afterSlash === undefined) return null;
123 +  return readBogusComment(input, pos, pos + 2, "Invalid end ta
    +g is treated as a comment", warnings);
124 +}
125 +
126  // Reads "<name ...>" or "</name ...>" starting at the "<".
127  // If the input ends inside the tag, the tag is kept with term
     inated: false.
 91 -function readTag(input, start) {
128 +function readTag(input, start, warnings) {
129    const isEndTag = input[start + 1] === "/";
130    let pos = start + (isEndTag ? 2 : 1);
131
```

● Я нормалізував вхід (regex безпечний, без зациклювання), замінив isTagStart на readMarkup, який визначає і читає вміст після <, а </> тепер повертається як токен "ignored". Далі перейду до warnings у тегах і атрибутах.

Read 1 file (ctrl+o to expand)
└ Found 5 new diagnostic issues in 1 file (ctrl+o to expand)

● Update(html2json.js)
└ Added 6 lines, removed 1 line

```
155        }
156        // A "/" not followed by ">" is ignored, as in browsers.
157      } else {
158 -      pos = readAttribute(input, pos, attributes);
158 +      pos = readAttribute(input, pos, attributes, warnings);
159      }
160    }
161
162 +  if (!terminated) {
163 +    const tagText = (isEndTag ? "</" : "<") + name;
164 +    warnings.push({ message: `Tag ${tagText}> is cut off by th
    +e end of input`, offset: start });
165 +  }
166 +
167    return {
168      type: isEndTag ? "endTag" : "startTag",
169      name,
```

● Update(html2json.js)
└ Added 1 line, removed 1 line

```
181
182  // Reads one attribute (name, optional "=" and value), adds it
      to `attributes`
183  // and returns the position right after it.
184 -function readAttribute(input, pos, attributes) {
184 +function readAttribute(input, pos, attributes, warnings) {
185    const nameStart = pos;
186    // The first character always belongs to the name, even "="
     (<div =a>).
```

```
    187    pos++;
```

● Update(html2json.js)
  ⎿ Added 4 lines
```
    199    let value;
    200
    201    if (quote === '"' || quote === "'") {
    202 +    // Like in browsers, the value runs to the next matching q
        +uote, wherever it is.
    203      const close = input.indexOf(quote, pos + 1);
    204 +    if (close === -1) {
    205 +      warnings.push({ message: `Unclosed quote in attribute "$
        +{name}"`, offset: pos });
    206 +    }
    207      const valueEnd = close === -1 ? input.length : close;
    208      value = input.slice(pos + 1, valueEnd);
    209      pos = close === -1 ? valueEnd : close + 1;
```

● Частина 2 готова: незакрита лапка тепер дає попередження з власною позицією, а обрізаний тег окремо, тож видно, де саме проблема. Далі займуся коментарями, doctype та bogus comments.

● Update(html2json.js)
  ⎿ Added 65 lines
```
    179    return isAsciiWhitespace(ch) || ch === "/" || ch === ">";
    180  }
    181
    182 +// Reads what starts with "<!": a comment, a doctype or a bogu
        +s comment.
    183 +function readMarkupDeclaration(input, start, warnings) {
    184 +  if (input.startsWith("--", start + 2)) return readComment(in
        +put, start, warnings);
    185 +  if (input.slice(start + 2, start + 9).toLowerCase() === "doc
        +type") {
    186 +    return readDoctype(input, start, warnings);
    187 +  }
    188 +  return readBogusComment(input, start, start + 2, `"<!" marku
        +p is treated as a comment`, warnings);
    189 +}
    190 +
    191 +/*
    192 +  Reads "<!-- ... -->". As in browsers, "--!>" also closes a c
        +omment, and
    193 +  "<!-->" and "<!--->" are empty comments. Only "--" is search
        +ed for, so the
    194 +  scan moves forward once even when "--!>" never appears in th
        +e input.
    195 +*/
    196 +function readComment(input, start, warnings) {
    197 +  const contentStart = start + 4;
    198 +  if (input[contentStart] === ">") return comment("", start, c
        +ontentStart + 1);
    199 +  if (input.startsWith("->", contentStart)) return comment("",
        + start, contentStart + 2);
    200 +
    201 +  let dashes = input.indexOf("--", contentStart);
    202 +  while (dashes !== -1) {
    203 +    if (input[dashes + 2] === ">") {
    204 +      return comment(input.slice(contentStart, dashes), start,
        + dashes + 3);
    205 +    }
    206 +    if (input.startsWith("!>", dashes + 2)) {
    207 +      return comment(input.slice(contentStart, dashes), start,
        + dashes + 4);
    208 +    }
    209 +    dashes = input.indexOf("--", dashes + 1);
    210 +  }
    211 +
    212 +  warnings.push({ message: "Comment is not closed before the e
        +nd of input", offset: start });
    213 +  return comment(input.slice(contentStart), start, input.lengt
        +h);
    214 +}
    215 +
    216 +// Reads "<!DOCTYPE ...>". `content` is the text after the key
        +word ("html").
```

```
217 +function readDoctype(input, start, warnings) {
218 +  const contentStart = start + "<!doctype".length;
219 +  const close = input.indexOf(">", contentStart);
220 +  if (close === -1) {
221 +    warnings.push({ message: "Doctype is not closed before the
    + end of input", offset: start });
222 +  }
223 +  const contentEnd = close === -1 ? input.length : close;
224 +  return {
225 +    type: "doctype",
226 +    content: trimAsciiWhitespace(input.slice(contentStart, con
    +tentEnd)),
227 +    start,
228 +    end: close === -1 ? contentEnd : close + 1,
229 +  };
230 +}
231 +
232 +/*
233 +  Markup browsers do not understand ("<?xml ?>", "<![CDATA[ ]]
    +>", "</ 3>")
234 +  becomes a comment up to the first ">", as in browsers.
235 +*/
236 +function readBogusComment(input, start, contentStart, message,
    + warnings) {
237 +  warnings.push({ message, offset: start });
238 +  const close = input.indexOf(">", contentStart);
239 +  const contentEnd = close === -1 ? input.length : close;
240 +  return comment(input.slice(contentStart, contentEnd), start,
    + close === -1 ? contentEnd : close + 1);
241 +}
242 +
243 +function comment(content, start, end) {
244 +  return { type: "comment", content, start, end };
245 +}
246 +
247  // Reads one attribute (name, optional "=" and value), adds it
       to `attributes`
248  // and returns the position right after it.
249  function readAttribute(input, pos, attributes, warnings) {
```

- Розбір частини 3 готовий: тут розібрано все, що йде після <!, і коментарі шукаються одним проходом, щоб уникнути квадратичного часу. Наступний крок — перевірка.

  Ran 1 shell command (ctrl+o to expand)
  └ Found 2 new diagnostic issues in 1 file (ctrl+o to expand)

- Усі кейси дали очікуваний результат. Додам нові рішення в README:

  Ran 2 shell commands (ctrl+o to expand)
- Етап 8 готовий: токенізатор тепер розбирає коментарі, doctype і решту розмітки після <!, <? і </, і на всіх перевірках результат правильний. Зараз ми на 8 з 15 етапів, приблизно 47%.

  Ще до етапу 8 зробив коміт fa3a04f з токенізатором, README і семплами. Етап 8 поки не закомічений: скажи, коли зберегти.

  Результати перевірки

| Вхід | Результат |
|---|---|
| <!---->, <!-->, <!---> | порожні коментарі, як у браузері |
| <!-- a ---> | " a –" |
| <!-- b --!> | закрився на --!> |
| <!-- -- inside --> | -- всередині не закриває коментар |
| <!--[if IE]>...<![endif]--> | один коментар |
| <!doctype html>, <!DOCTYPE html PUBLIC ...> | "html" і весь рядок після DOCTYPE |
| <![CDATA[ a ]]>, <?xml | bogus comments + warnings |

| | |
|---|---|
| ...?>, <!foo> | |
| a</>b</ 3>c | "a", "b", коментар " 3", "c" + warnings |
| text</ | текст "text</" |
| <div class="a>текст | class = "a>текст " + 2 warnings: про лапку (позиція 11) і про обрізаний тег (позиція 0) |
| <!, <!DOCTYPE html (обрив) | без падіння, з warning |

На семплах:
- Нічого не впало. Warnings є тільки там, де вони очікувані: у 08 (CDATA і <?xml), 11 і 12.
- Швидкодія та сама, що й на етапі 7: 1,5 МБ і 10 000 коментарів обробляються за 19 мс. Тобто пошук через -- справді дав лінійний час.
- У 13-eof-in-script поки 0 warnings. Це нормально, бо про незакритий <script> має сказати tree builder: це він побачить, що end-тегу не було.

Що ще змінилось

- Нормалізація входу. normalizeInput (BOM, \r\n → \n) уже є, але викликати її html2json почне на етапі 10.
- Порядок warnings. Warnings токенізатора йдуть у спільний список { message, offset }. На етапі 10 туди ж писатиме tree builder, і в кінці список відсортуємо за offset.
- README. Додав --!>, bogus comments, </> і пояснення, чому незакриту лапку ми не намагаємося «вгадати».
- Переноси в html2json.js знову привів до CRLF.

Наступний етап 9: декодування entities. Там треба буде розділити script/style (вміст лишається як є) і textarea/title (entities декодуються) і вирішити, який набір іменованих entities брати.

✻ Churned for 3m 8s · done 6:48 PM

※ recap: Робимо тестове html2json: свій HTML-парсер без DOMParser, і токенізатор з коментарями й doctype уже готовий (етап 8 з 15, ~47%). Далі етап 9, декодування entities, але спершу скажи, чи комітити етап 8.

❯ ок, етап 8 норм, комить.

пару питань:
1. ми ж домовлялись без regex, а в normalizeInput є /\r\n?/g. чому тут можна?
2. в README warnings описані як { message, line, column }, а в коді { message, offset }. це потім зміниться чи README треба поправити?

по етапу 9:
- які іменовані entities брати? всі 2000 це ж забагато, ні?
- а шо буде якщо в html написати якесь криве число, типу &#99999999999; або &#0;? не впаде?
- &copy без ; треба декодувати чи ні?
- textarea і title ми ж казали шо там entities декодуються, а в script/style ні. не забудь розділити

давай етап 9

Ran 1 shell command (ctrl+o to expand)

● Коміт 23003cd зроблено. Тепер відповіді.

1. Regex у normalizeInput

Ми домовлялися не про «жодних regex», а про жодних regex, які можуть зависнути (catastrophic backtracking). У README так і записано: «no regular expressions that could backtrack catastrophically». Але ти маєш рацію, що в розмові я сформулював це нечітко: «без рекурсії та regex».

Небезпечні regex мають вкладені або сусідні повторення, які перекривають одне одного, наприклад (a+)+ чи (\s*)*x. На вході без збігу рушій перебирає експоненційну кількість способів розбити рядок. /\r\n?/g — це фіксований символ і один необов'язковий символ. Варіантів збігу в кожній позиції максимум два, тому регулярка йде одним лінійним проходом і зависнути не може.

Можна переписати це циклом, але regex тут коротший і зрозуміліший. Я б лишив його і так само пояснив на рев'ю.

2. { message, offset } і { message, line, column }

README правильний. Він описує вихід html2json, а { message, offset } — внутрішній формат. На етапі 10 html2json перетворить кожен offset на line/column через бінарний пошук, перш ніж повернути результат.

---

Етап 9: рішення

Які іменовані entities брати. Повний список містить 2231 запис. Його неможливо набрати руками, лише згенерувати з офіційного entities.json, і він додав би у файл ~40 КБ. Я беру ~55 найуживаніших:
- XML-базові amp lt gt quot apos
- пробіли nbsp ensp emsp thinsp
- типографіку: лапки, тире, hellip, bull, middot
- символи copy reg trade deg plusmn times divide, дроби, валюти, стрілки

Будь-який інший символ автор може записати числом (∞). Розширити підтримку означає просто додати рядок у таблицю.

Криві числа. Найнебезпечніше місце етапу: String.fromCodePoint(99999999999) кидає RangeError, тобто парсер упав би на валідному вході. Робимо як у специфікації:
- &#0;, числа більші за 0x10FFFF і половинки сурогатних пар (&#xD800;) замінюємо на U+FFFD (&#xfffd;) з warning
- €–Ÿ перетворюємо за таблицею Windows-1252: € дає €. Так робить браузер, бо старі сайти писали ці коди саме так
- число, поки читаємо цифри, обмежуємо зверху. Тож &#999…999; з мільйоном цифр не переповниться
- &#; і &#x; (без цифр) не вважаються entity і лишаються текстом
- &#169 без ; декодується, як у браузері

&copy без ;. Специфікація дозволяє обходитися без ; тільки для ~100 «старих» імен з HTML 3.2 (amp lt gt quot nbsp copy reg …). Ми робимо так само:
- &copy 2024 дає © 2024, &amp done дає & done
- &copyright дає ©right, бо так поводиться браузер (довший префікс, який збігся зі старим іменем)
- в атрибутах старе ім'я без ; не декодується, якщо далі йде = або буква чи цифра. Причина: URL ?a=1&copy=3 інакше зламався б, бо &copy= — це параметр, а не символ
- &apos, &hellip без ; не декодуються: це не старі імена

Невідомі &name; лишаються як є. Для &notit; браузер видасть ¬it; (префікс not), але це вже вгадування. Ми просто пишемо це в обмеження.

Таблиця entities в Map, а не в {}. Це та сама історія з __proto__: якби таблиця була звичайним об'єктом, &constructor; знайшов би в прототипі функцію і вставив би її в текст.

Розділення raw text:
- script, style: вміст як є, без декодування
- textarea, title: декодуємо
- текст і значення атрибутів: декодуємо
- коментарі, doctype, імена атрибутів: не декодуємо

Частина 1: таблиці

Read 1 file (ctrl+o to expand)

● Таблиці й декодування винесу в окремий розділ після токенізатора, щоб токенізатор лишився читабельним.

● Update(html2json.js)
  ⎿ Added 2 lines
```
20
21  // No tags inside: everything up to the matching end tag is tex
    t.
22  const RAW_TEXT_TAGS = new Set(["script", "style", "textarea", "
    title"]);
23 +// Of those, character references are decoded only here, as in
   +browsers.
24 +const DECODED_RAW_TEXT_TAGS = new Set(["textarea", "title"]);
```

```
     25
     26  function isAsciiWhitespace(ch) {
     27    return ch === " " || ch === "\t" || ch === "\n" || ch === "\r
         " || ch === "\f";

  Ran 1 shell command (ctrl+o to expand)
    └ Found 4 new diagnostic issues in 1 file (ctrl+o to expand)

● Update(html2json.js)
    └ Added 42 lines
     307    return input.length;
     308  }
     309
     310 +/* ---------- Character references (entities) ---------- */
     311 +
     312 +// The most common named references. The full HTML list has ov
         +er 2000 names,
     313 +// any character can also be written as a number (&#8734;).
     314 +// A Map, not an object: with {} "&constructor;" would find Ob
         +ject.prototype.constructor.
     315 +const NAMED_REFERENCES = new Map([
     316 +  ["amp", "&"], ["lt", "<"], ["gt", ">"], ["quot", '"'], ["apo
         +s", "'"],
     317 +  ["AMP", "&"], ["LT", "<"], ["GT", ">"], ["QUOT", '"'],
     318 +  ["nbsp", " "], ["ensp", " "], ["emsp", " "], ["thinsp", " "]
         +, ["shy", ""],
     319 +  ["copy", "©"], ["COPY", "©"], ["reg", "®"], ["REG", "®"], ["
         +trade", "™"],
     320 +  ["deg", "°"], ["plusmn", "±"], ["times", "×"], ["divide", "÷
         +"], ["minus", "−"],
     321 +  ["ne", "≠"], ["le", "≤"], ["ge", "≥"], ["infin", "∞"], ["mic
         +ro", "µ"],
     322 +  ["frac14", "¼"], ["frac12", "½"], ["frac34", "¾"], ["sup1",
         +"¹"], ["sup2", "²"], ["sup3", "³"],
     323 +  ["middot", "·"], ["bull", "•"], ["hellip", "…"], ["para", "¶
         +"], ["sect", "§"],
     324 +  ["ndash", "–"], ["mdash", "—"], ["iexcl", "¡"], ["iquest", "
         +¿"],
     325 +  ["laquo", "«"], ["raquo", "»"], ["lsquo", "‘"], ["rsquo", "’
         +"], ["sbquo", ","],
     326 +  ["ldquo", "“"], ["rdquo", "”"], ["bdquo", "„"],
     327 +  ["cent", "¢"], ["pound", "£"], ["yen", "¥"], ["euro", "€"],
         +["curren", "¤"],
     328 +  ["larr", "←"], ["rarr", "→"], ["uarr", "↑"], ["darr", "↓"],
         +["harr", "↔"], ["hearts", "♥"],
     329 +]);
     330 +
     331 +// Old names (from HTML 3.2) that browsers also decode without
         + the ";" ("&copy 2024").
     332 +const LEGACY_NAMED_REFERENCES = new Set([
     333 +  "amp", "lt", "gt", "quot", "AMP", "LT", "GT", "QUOT", "nbsp"
         +, "shy",
     334 +  "copy", "COPY", "reg", "REG", "deg", "plusmn", "times", "div
         +ide", "micro",
     335 +  "frac14", "frac12", "frac34", "sup1", "sup2", "sup3", "middo
         +t", "para", "sect",
     336 +  "iexcl", "iquest", "laquo", "raquo", "cent", "pound", "yen",
         + "curren",
     337 +]);
     338 +const LONGEST_LEGACY_NAME = 6;
     339 +
     340 +// Browsers read &#128;–&#159; as Windows-1252 bytes, because
         +old pages used them that way.
     341 +const WINDOWS_1252_REPLACEMENTS = new Map([
     342 +  [0x80, 0x20ac], [0x82, 0x201a], [0x83, 0x0192], [0x84, 0x201
         +e], [0x85, 0x2026],
     343 +  [0x86, 0x2020], [0x87, 0x2021], [0x88, 0x02c6], [0x89, 0x203
         +0], [0x8a, 0x0160],
     344 +  [0x8b, 0x2039], [0x8c, 0x0152], [0x8e, 0x017d], [0x91, 0x201
         +8], [0x92, 0x2019],
     345 +  [0x93, 0x201c], [0x94, 0x201d], [0x95, 0x2022], [0x96, 0x201
         +3], [0x97, 0x2014],
     346 +  [0x98, 0x02dc], [0x99, 0x2122], [0x9a, 0x0161], [0x9b, 0x203
         +a], [0x9c, 0x0153],
     347 +  [0x9e, 0x017e], [0x9f, 0x0178],
     348 +]);
```

```
349 +
350 +const MAX_CODE_POINT = 0x10ffff;
351 +
352  function showExample1() {
353    const htmlExample = `<!DOCTYPE html>
354  <html lang="en">
```

● Я обчислю довжину найдовшої старої назви автоматично, щоб вона не розходилася з таблицею, якщо її колись розширять.

● Update(html2json.js)
  ⎿ Added 1 line, removed 1 line

```
335    "frac14", "frac12", "frac34", "sup1", "sup2", "sup3", "middo
     t", "para", "sect",
336    "iexcl", "iquest", "laquo", "raquo", "cent", "pound", "yen",
      "curren",
337  ]);
338 -const LONGEST_LEGACY_NAME = 6;
338 +const LONGEST_LEGACY_NAME = Math.max(...[...LEGACY_NAMED_REFER
    +ENCES].map((name) => name.length));
339
340  // Browsers read &#128;-&#159; as Windows-1252 bytes, because
     old pages used them that way.
341  const WINDOWS_1252_REPLACEMENTS = new Map([
```

● Ця частина пояснює вибір структур даних: які теги декодують текст, чому іменовані посилання зберігаються в Map з урахуванням регістру, які імена дозволені без ;, та що в таблиці Windows-1252 27 записів. Далі йде частина про декодування.

● Update(html2json.js)
  ⎿ Added 107 lines

```
349
350  const MAX_CODE_POINT = 0x10ffff;
351
352 +function isAsciiDigit(ch) {
353 +  return ch >= "0" && ch <= "9";
354 +}
355 +
356 +function isAsciiHexDigit(ch) {
357 +  return isAsciiDigit(ch) || (ch >= "a" && ch <= "f") || (ch >
    += "A" && ch <= "F");
358 +}
359 +
360 +function isAsciiAlphanumeric(ch) {
361 +  return isAsciiLetter(ch) || isAsciiDigit(ch);
362 +}
363 +
364 +/*
365 +  Replaces character references in `text`: "&amp;", "&#169;",
    +"&#xA9;".
366 +  Unknown or broken references are kept as written. `offset` i
    +s the position of
367 +  `text` in the input, used for warnings. In attribute values
    +a legacy name
368 +  without ";" is not decoded before "=" or a letter/digit, so
    +URLs like
369 +  "?a=1&copy=2" stay intact.
370 +*/
371 +function decodeCharacterReferences(text, inAttribute, offset,
    +warnings) {
372 +  let amp = text.indexOf("&");
373 +  if (amp === -1) return text;
374 +
375 +  const parts = [];
376 +  let copied = 0;
377 +  while (amp !== -1) {
378 +    const reference = text[amp + 1] === "#"
379 +      ? readNumericReference(text, amp, offset, warnings)
380 +      : readNamedReference(text, amp, inAttribute);
381 +
382 +    if (reference === null) {
383 +      amp = text.indexOf("&", amp + 1);
384 +    } else {
385 +      parts.push(text.slice(copied, amp), reference.value);
386 +      copied = reference.end;
```

```
387 +      amp = text.indexOf("&", copied);
388 +    }
389 +  }
390 +  parts.push(text.slice(copied));
391 +  return parts.join("");
392 +}
393 +
394 +// Reads "&#169;" or "&#xA9;" (the ";" is optional, as in brow
    +sers).
395 +// Returns { value, end } or null if there are no digits ("&#;
    +").
396 +function readNumericReference(text, amp, offset, warnings) {
397 +  let pos = amp + 2;
398 +  const isHex = text[pos] === "x" || text[pos] === "X";
399 +  if (isHex) pos++;
400 +
401 +  const digitsStart = pos;
402 +  let codePoint = 0;
403 +  while (pos < text.length && (isHex ? isAsciiHexDigit(text[po
    +s]) : isAsciiDigit(text[pos]))) {
404 +    // Capped, so a very long number cannot grow past the vali
    +d range.
405 +    codePoint = Math.min(codePoint * (isHex ? 16 : 10) + parse
    +Int(text[pos], 16), MAX_CODE_POINT + 1);
406 +    pos++;
407 +  }
408 +  if (pos === digitsStart) return null;
409 +  if (text[pos] === ";") pos++;
410 +
411 +  return { value: codePointToString(codePoint, text.slice(amp,
    + pos), offset + amp, warnings), end: pos };
412 +}
413 +
414 +/*
415 +  Turns a number from a reference into a string, as browsers d
    +o: 0, values above
416 +  U+10FFFF and surrogates become U+FFFD, and 0x80–0x9F use Win
    +dows-1252.
417 +  String.fromCodePoint would throw on the out-of-range values.
418 +*/
419 +function codePointToString(codePoint, source, offset, warnings
    +) {
420 +  const isSurrogate = codePoint >= 0xd800 && codePoint <= 0xdf
    +ff;
421 +  if (codePoint === 0 || codePoint > MAX_CODE_POINT || isSurro
    +gate) {
422 +    warnings.push({ message: `Invalid character reference "${s
    +ource}" is replaced with U+FFFD`, offset });
423 +    return "\uFFFD";
424 +  }
425 +  if (WINDOWS_1252_REPLACEMENTS.has(codePoint)) {
426 +    return String.fromCodePoint(WINDOWS_1252_REPLACEMENTS.get(
    +codePoint));
427 +  }
428 +  return String.fromCodePoint(codePoint);
429 +}
430 +
431 +/*
432 +  Reads "&name;". Without ";" only legacy names are decoded, a
    +nd the longest
433 +  legacy prefix wins, as in browsers ("&copyright" -> "©right"
    +).
434 +  Unknown "&name;" is not decoded: guessing a prefix there cou
    +ld be wrong.
435 +  Returns { value, end } or null.
436 +*/
437 +function readNamedReference(text, amp, inAttribute) {
438 +  let pos = amp + 1;
439 +  while (pos < text.length && isAsciiAlphanumeric(text[pos]))
    +pos++;
440 +  const name = text.slice(amp + 1, pos);
441 +  if (name === "") return null;
442 +
443 +  if (text[pos] === ";") {
444 +    return NAMED_REFERENCES.has(name) ? { value: NAMED_REFEREN
    +CES.get(name), end: pos + 1 } : null;
```

```
445 +  }
446 +
447 +  for (let length = Math.min(name.length, LONGEST_LEGACY_NAME)
    +; length >= 2; length--) {
448 +    const prefix = name.slice(0, length);
449 +    if (!LEGACY_NAMED_REFERENCES.has(prefix)) continue;
450 +
451 +    const end = amp + 1 + length;
452 +    const next = text[end];
453 +    if (inAttribute && (next === "=" || isAsciiAlphanumeric(ne
    +xt))) return null;
454 +    return { value: NAMED_REFERENCES.get(prefix), end };
455 +  }
456 +  return null;
457 +}
458 +
459  function showExample1() {
460    const htmlExample = `<!DOCTYPE html>
461  <html lang="en">
```

● Що робить частина 2:
  – decodeCharacterReferences — головна функція. Якщо в тексті немає &, рядок повертається одразу, без жодної роботи. Так буде з більшістю текстів. Якщо & є, стрибаємо між ними через indexOf, шматки збираємо в масив parts, а в кінці робимо один join. Так копіювання лишається лінійним у будь-якому рушії JS.
  – readNumericReference читає цифри одну за одною і обмежує значення через Math.min(..., MAX_CODE_POINT + 1). Що б не прийшло на вхід, число не росте далі за 0x110000. &#99999999999; перетвориться на 0x110000, а потім на U+FFFD, без жодного RangeError і без мільярдів. parseInt(ch, 16) підходить і для десяткових цифр, бо 0–9 у шістнадцятковій системі ті самі.
  – codePointToString — єдине місце, де викликається String.fromCodePoint, і сюди не дійде жодне значення, на якому вона кидає помилку. Warning видається лише для замін на U+FFFD: це єдиний випадок, коли дані справді губляться.
  – readNamedReference:
    – &name; декодується, лише якщо name точно є в таблиці
    – без ; шукаємо найдовший старий префікс, від 6 символів до 2
    – в атрибуті не декодуємо, якщо далі йде = або буква чи цифра
  – Нескінченний цикл неможливий: після кожної ітерації позиція пошуку & строго росте.

  Частина 3: підключення до токенізатора

● Update(html2json.js)
  ⎿  Added 5 lines, removed 4 lines

```
66    let pos = 0;
67    let textStart = 0;
68
69 -  function pushText(end) {
70 -    if (end > textStart) {
71 -      tokens.push({ type: "text", content: input.slice(textStar
   -t, end), start: textStart, end });
72 -    }
69 +  function pushText(end, decode) {
70 +    if (end <= textStart) return;
71 +    const raw = input.slice(textStart, end);
72 +    const content = decode ? decodeCharacterReferences(raw, fal
   +se, textStart, warnings) : raw;
73 +    tokens.push({ type: "text", content, start: textStart, end
   +});
74    }
75
76    while (pos < input.length) {
```

● Update(html2json.js)
  ⎿  Added 3 lines, removed 3 lines

```
83        continue;
84      }
85
86 -    pushText(lt);
86 +    pushText(lt, true);
87      if (token.type !== "ignored") tokens.push(token);
88      pos = token.end;
89
...
 92        if (RAW_TEXT_TAGS.has(name)) {
```

```
 93          textStart = pos;
 94          pos = findRawTextEnd(input, pos, name);
 95 -        pushText(pos);
 95 +        pushText(pos, DECODED_RAW_TEXT_TAGS.has(name));
 96        }
 97      }
 98      textStart = pos;
 99    }
100
101 -  pushText(input.length);
101 +  pushText(input.length, true);
102    return tokens;
103  }
104
```

● Update(html2json.js)
  ⎿ Added 2 lines, removed 2 lines

```
273        warnings.push({ message: `Unclosed quote in attribute "${name}"`, offset: pos });
274      }
275      const valueEnd = close === -1 ? input.length : close;
276 -    value = input.slice(pos + 1, valueEnd);
276 +    value = decodeCharacterReferences(input.slice(pos + 1, valueEnd), true, pos + 1, warnings);
277      pos = close === -1 ? valueEnd : close + 1;
278    } else {
279      // Unquoted: ends at whitespace or ">". A "/" is part of the value (href=/a/).
280      const valueStart = pos;
281      while (pos < input.length && !isAsciiWhitespace(input[pos]) && input[pos] !== ">") pos++;
282 -    value = input.slice(valueStart, pos);
282 +    value = decodeCharacterReferences(input.slice(valueStart, pos), true, valueStart, warnings);
283    }
284
285    attributes.push({ name, value });
```

● Update(html2json.js)
  ⎿ Added 3 lines, removed 2 lines

```
58      { type: "text",     content, start, end }
59      { type: "comment",  content, start, end }
60      { type: "doctype",  content, start, end }
61 -  `start` and `end` are offsets in the input. Problems found in the input are
62 -  added to `warnings` as { message, offset }.
61 +  Character references in text and attribute values are decoded, except in
62 +  <script> and <style>. `start` and `end` are offsets in the input. Problems
63 +  found in the input are added to `warnings` as { message, offset }.
64   */
65  function tokenize(input, warnings = []) {
66    const tokens = [];
```

  Ran 1 shell command (ctrl+o to expand)
  ⎿ Found 2 new diagnostic issues in 1 file (ctrl+o to expand)

● Тести всі проходять. Я помітив, що warning для &# з 100 000 цифрами виводить усі цифри, тож його треба обрізати, і зараз я цим займуся.

● Update(html2json.js)
  ⎿ Added 2 lines, removed 1 line

```
421  function codePointToString(codePoint, source, offset, warnings) {
422    const isSurrogate = codePoint >= 0xd800 && codePoint <= 0xdfff;
423    if (codePoint === 0 || codePoint > MAX_CODE_POINT || isSurrogate) {
424 -    warnings.push({ message: `Invalid character reference "${source}" is replaced with U+FFFD`, offset });
424 +    const shown = source.length > 20 ? `${source.slice(0, 20)}...` : source;
425 +    warnings.push({ message: `Invalid character reference "${shown}" is replaced with U+FFFD`, offset });
```

```
    426       return "\uFFFD";
    427     }
    428     if (WINDOWS_1252_REPLACEMENTS.has(codePoint)) {
```

Ran 1 shell command (ctrl+o to expand)
└ Found 2 new diagnostic issues in 1 file (ctrl+o to expand)

- Тепер README:

Ran 1 shell command (ctrl+o to expand)

- Етап 9 готовий: entities декодуються як у браузері, і на кривих числах нічого не падає. Зараз ми на 9 з 15 етапів, приблизно 55%.

Коміт для етапу 9 ще не робив: скажи, коли зберегти.

Результати перевірки

| Вхід | Результат |
|---|---|
| & < © & / &Amp; | & < © & / &Amp; лишився, бо такого імені немає |
| © © © &#169 x | © © © © x |
| &#0; &#99999999999; &#xFFFFFFFF; &#xD800; &#1114112; | 5 × "\uFFFD" у всіх п'яти + 5 warnings, без падіння |
| &# + 100 000 дев'яток | "\uFFFD", warning обрізаний до 20 символів |
| &#; &#x; &#xG; & alone &; | лишились як є |
| € – Ÿ | € — Ÿ (Windows-1252) |
| &copy 2024 &amp done &copyright | © 2024 & done ©right |
| &apos x &hellip x | не декодуються, бо це не старі імена |
| &constructor; &__proto__; &toString; | лишились як є, тобто Map спрацював |
| href="?a=1&b=2&copy=3©&copy x&reg2" | ?a=1&b=2&copy=3©© x&reg2: &copy= і &reg2 в атрибуті не зачеплені |
| <script>&</script>, <style><</style> | лишились як є |
| <textarea>& <b></textarea>, <title>A & B</title> | & <b>, A & B |
| коментар, doctype, ім'я атрибута з & | не декодуються |

На всіх 23 семплах нічого не впало. Нові warnings з'явилися тільки в 05-text-and-entities.html (&#0; і &#xFFFFFFFF;), і там вони очікувані. Швидкодія не змінилась: 1,5 МБ обробляється за ~22 мс.

Що ще змінилось

- README: переписав розділ про entities. В обмеження додав &notit;: браузер декодує старий префікс навіть з ;, а ми лишаємо таке як є.
- У файлі тепер є не-ASCII символи (©, —, € у таблиці), тож кодування стало UTF-8. index.html має <meta charset="UTF-8">, а скрипт без атрибута charset береться в кодуванні сторінки, тож усе працює. Невидимі символи (nbsp, shy) записані як \uXXXX.

Нюанс для етапу 10

Ми вирішили викидати whitespace-only текст з \n. Але після декодування 
 перетвориться на "\n" і теж зникне, хоча автор явно написав цей символ. Тому перевіряти «тільки пробіли» треба по оригінальному тексту, а не по декодованому. У токені є start/end, тож tree builder зможе це зробити.

Наступний етап 10: tree builder. Там стек, void-теги, , неявне закриття, SVG-регістр, Object.create(null) для атрибутів, правила для пробілів, warnings з line/column, і нарешті справжній html2json.

✻ Cogitated for 3m 25s · done 7:05 PM
※ recap: Пишемо свій HTML-парсер html2json для тестового: токенізатор і декодування entities готові, це етап 9 з 15 (приблизно 55%). Далі треба закомітити етап 9 і почати етап 10, tree builder.

❯ ок, комить етап 9.

пару питань перед етапом 10:
1. я не до кінця поняв про Map і &constructor. можеш на простому прикладі показати шо б сталось якби було {}?
2. про 
 не дуже поняв, тобто ми будемо перевіряти "тільки пробіли" по тексту до декодування? як це буде в коді?
3. якщо я зараз нажму Convert в index.html, вже шось нове побачу?

ти кажеш:Ми вирішили викидати whitespace-only текст з \n. Але після декодування 
 перетвориться на "\n" і теж зникне, хоча автор явно написав цей символ. Тому перевіряти «тільки пробіли» треба по оригінальному тексту, а не по декодованому. У токені є start/end, тож tree builder зможе це зробити. — чи не стане нам це боком згодом? залишаємо так чи краще виправити? просто не до кінця поняв чому ти на цьому наголошуєш

і давай етап 10. поясни як працює стек, покажи на example 2 по кроках шо в ньому лежить

Ran 1 shell command (ctrl+o to expand)
⎿ PostToolUse:Bash says: Tip: Run /ultrareview before you push to catch bugs with a cloud-based multi-agent review — 3 free reviews left.

● Коміт d542906 готовий. Далі перевірю, чи JSON.stringify витримає семпл із 10 000 вкладених , бо це може зламати index.html.

Ran 1 shell command (ctrl+o to expand)

● Знайшов серйозну проблему: JSON.stringify рекурсивний і падає вже на 10 000 рівнів вкладеності, тож кнопка Convert впаде на 22-deep-nesting.html. Наступним кроком знайду точний поріг.

Ran 1 shell command (ctrl+o to expand)

● Поріг у Node близько 2800 рівнів, а в різних браузерах він різний. Це рішення варто прийняти до того, як писати tree builder, тому спершу відповім на твої питання і поясню стек, а потім запитаю.

1. {} проти Map на прикладі

Ось що я отримав у Node:
```
const table = { amp: "&", copy: "©" };     // звичайний об'єкт

"constructor" in table     // true, хоча ми такого не додавали
table["constructor"]       // function Object() { [native code] }

// і парсер для тексту "a &constructor; b" видав би:
"a function Object() { [native code] } b"
```
Будь-який {} «бачить» властивості свого прототипу Object.prototype: constructor, toString, hasOwnProperty, valueOf… Тому перевірка name in table для &constructor; або &toString; каже «так, є», і в текст потрапляє функція, перетворена на рядок. Map зберігає тільки те, що в нього поклали: map.has("constructor") дає false.

2 і «чи не стане боком»: 


Наголошую на цьому, бо це тихий баг. Нічого не падає, warning немає, просто з результату зникає символ, який автор написав навмисно. Такі баги на рев'ю найнеприємніші. Виправити його легко, тож виправляємо зараз.

Як це буде в коді: токенізатор, коли створює text-токен, ще до декодування дивиться на сирий текст і ставить прапорець:
```
{ type: "text", content: "\n", formattingWhitespace: false, ... }   // з <div>&#10;</div>
{ type: "text", content: "\n  ", formattingWhitespace: true, ... }  // відступ між тегами
```

formattingWhitespace: true означає, що в сирому тексті є тільки [ \t\n\r\f] і хоча б один \n. Tree builder викидає текст лише з цим прапорцем, якщо ми не всередині pre/textarea. Факт про вхід визначає токенізатор (він бачить сирий текст), а рішення ухвалює tree builder (він знає, чи ми в <pre>). Це той самий поділ, що й з атрибутами.

3. Convert в index.html

Поки що ні, ти побачиш стару заглушку. html2json ще не викликає парсер, і це якраз робота етапу 10. Після нього Convert покаже справжнє дерево.

---

Як працює стек

Стек — це шлях від кореня до місця, куди зараз додаються вузли. Вершина стеку (останній елемент) — це «поточний вузол»: усе нове йде в його children.
- start-тег: створюємо вузол, додаємо його в children вершини, кладемо в стек. Void-тег і <x/> у стек не кладемо, бо в них не буде дітей.
- текст, коментар, doctype: додаємо в children вершини, стек не змінюється.
- end-тег: шукаємо в стеку зверху вниз елемент з таким іменем. Знайшли — знімаємо все до нього включно, не знайшли — ігноруємо + warning.
- кінець входу: усе, що лишилося в стеку над root, незакрите. Для , <li> тощо це нормально, бо їхній end-тег необов'язковий. Для решти даємо warning.

Example 2 по кроках

```
<div>
<p>Hello world!</p>
  <button>Click me!</button>
  <textarea>Some very ... long string.</textarea>
</div>
```

| # | Токен | Дія | Стек після |
|---|---|---|---|
| 1 |  | додати в root, покласти в стек | [root, div] |
| 2 | "\n" | formattingWhitespace, тож пропустити | [root, div] |
| 3 |  | додати в div, покласти в стек | [root, div, p] |
| 4 | "Hello world!" | додати в p | [root, div, p] |
| 5 |  | p на вершині, зняти | [root, div] |
| 6 | "\n  " | пропустити | [root, div] |
| 7 | <button> | додати в div, покласти в стек | [root, div, button] |
| 8 | "Click me!" | додати в button | [root, div, button] |
| 9 | </button> | зняти button | [root, div] |
| 10 | "\n  " | пропустити | [root, div] |
| 11 | <textarea> | додати в div, покласти в стек. Токенізатор перейшов у raw text | [root, div, textarea] |
| 12 | "Some very ..." | додати в textarea | [root, div, textarea] |
| 13 | </textarea> | зняти textarea | [root, div] |
| 14 | "\n" | пропустити | [root, div] |
| 15 |  | зняти div | [root] |
| 16 | "\n" | пропустити | [root] |
| — | кінець | у стеку тільки root, warnings | [root] |

| | | немає | |
|---|---|---|---|

Дерево будується без рекурсії: стек — це звичайний масив. Вкладеність хоч у мільйон рівнів дасть просто масив на мільйон елементів, а не мільйон викликів функції.

Як це виглядає з помилкою: x

| Токен | Дія | Стек |
|---|---|---|
|  | | [root, b] |
|  | | [root, b, i] |
| "x" | у i | [root, b, i] |
|  | b знайдено нижче вершини: зняти i (warning:  не закритий) і b | [root] |
|  | i в стеку немає, ігнорувати (warning) | [root] |

---

Проблема з JSON.stringify

Заміряв у Node 26: JSON.stringify рекурсивний і кидає RangeError: Maximum call stack size exceeded на глибині ~2800 елементів. У браузерах поріг інший, але теж кінцевий. Тобто html2json поверне дерево з 22-deep-nesting.html без помилок, а convertHtml2JsonAndSet в index.html впаде на рядку з JSON.stringify. Те саме станеться в рев'юера, якщо він викличе JSON.stringify на нашому результаті у своєму тесті.

Наскільки я знаю, браузери теж обмежують глибину: Chrome при парсингу HTML не вкладає елементи глибше 512 рівнів, а глибші робить сусідами. Так що для варіанту з лімітом є прецедент.

● User answered Claude's questions:
  ⎿ · Як розв'язуємо падіння JSON.stringify на дуже глибокій вкладеності (~2800+ рівнів)? → Ліміт глибини (Recommended)

● Ліміт вкладеності беремо 512, як у Chrome, щоб результат можна було серіалізувати будь-де. Далі сплющені end-теги тихо закриватиму, а перший \n після <pre> і <textarea> браузер відкидатиме, починаючи з токенізатора.

● Update(html2json.js)
  ⎿ Added 8 lines, removed 2 lines
```
70    function pushText(end, decode) {
71      if (end <= textStart) return;
72      const raw = input.slice(textStart, end);
73 -    const content = decode ? decodeCharacterReferences(raw, fal
   -se, textStart, warnings) : raw;
74 -    tokens.push({ type: "text", content, start: textStart, end
   -});
73 +    tokens.push({
74 +      type: "text",
75 +      content: decode ? decodeCharacterReferences(raw, false, t
   +extStart, warnings) : raw,
76 +      // Checked before decoding, so "&#10;" written on purpose
   + is not taken for indentation.
77 +      formattingWhitespace: isFormattingWhitespace(raw),
78 +      start: textStart,
79 +      end,
80 +    });
81    }
82
83    while (pos < input.length) {
```

● Update(html2json.js)
  ⎿ Added 10 lines
```
44    return text.slice(start, end);
45  }
```

```
      46
      47 +// True for text of only ASCII whitespace with a line break: in
         +dentation between tags.
      48 +function isFormattingWhitespace(text) {
      49 +  let hasLineBreak = false;
      50 +  for (let i = 0; i < text.length; i++) {
      51 +    if (!isAsciiWhitespace(text[i])) return false;
      52 +    if (text[i] === "\n") hasLineBreak = true;
      53 +  }
      54 +  return hasLineBreak;
      55 +}
      56 +
      57  // Input preprocessing from the HTML spec: drop a leading BOM,
      58  // turn "\r\n" and a lone "\r" into "\n".
      59  function normalizeInput(input) {
```

● Update(html2json.js)
  ⎿ Added 1 line, removed 1 line

```
      65    Splits the input into a flat list of tokens in a single forwa
          rd pass:
      66      { type: "startTag", name, attributes, selfClosing, terminat
          ed, start, end }
      67      { type: "endTag",   name, attributes, selfClosing, terminat
          ed, start, end }
      68 -    { type: "text",     content, start, end }
      68 +    { type: "text",     content, formattingWhitespace, start, e
         +nd }
      69      { type: "comment",  content, start, end }
      70      { type: "doctype",  content, start, end }
      71    Character references in text and attribute values are decoded
          , except in
```

● Частина 1 пояснює, що isFormattingWhitespace перевіряє сирий текст до декодування й зупиняється на першому непробільному символі. Токенізатор лише ставить прапорець, а рішення щодо тексту ухвалює tree builder, який знає про <pre>. Наступний крок — частина 2 про константи tree builder.

  Read 1 file, ran 1 shell command (ctrl+o to expand)

● Update(html2json.js)
  ⎿ Added 37 lines

```
      475    return null;
      476  }
      477
      478 +/* ---------- Tree builder ---------- */
      479 +
      480 +const VOID_ELEMENTS = new Set([
      481 +  "area", "base", "br", "col", "embed", "hr", "img", "input",
          +"link", "meta", "source", "track", "wbr",
      482 +]);
      483 +
      484 +// A start tag of these closes a <p> that is the current eleme
          +nt: <p>text<div>.
      485 +const CLOSES_P = new Set([
      486 +  "address", "article", "aside", "blockquote", "dd", "details"
          +, "dialog", "div", "dl", "dt",
      487 +  "fieldset", "figcaption", "figure", "footer", "form", "h1",
          +"h2", "h3", "h4", "h5", "h6",
      488 +  "header", "hgroup", "hr", "li", "main", "menu", "nav", "ol",
          + "p", "pre", "section", "table", "ul",
      489 +]);
      490 +
      491 +// A start tag closes these while one of them is the current e
          +lement: <li>a<li>b.
      492 +const IMPLICITLY_CLOSES = new Map([
      493 +  ["li", new Set(["li"])],
      494 +  ["dt", new Set(["dt", "dd"])],
      495 +  ["dd", new Set(["dt", "dd"])],
      496 +  ["option", new Set(["option"])],
      497 +  ["optgroup", new Set(["option", "optgroup"])],
      498 +  ["tr", new Set(["td", "th", "tr"])],
      499 +  ["td", new Set(["td", "th"])],
      500 +  ["th", new Set(["td", "th"])],
      501 +]);
      502 +
      503 +// Their end tag may be left out in valid HTML, so closing the
```

```
    +m implicitly is not a warning.
504 +const OPTIONAL_END_TAGS = new Set([
505 +  "html", "head", "body", "p", "li", "dt", "dd", "option", "op
    +tgroup",
506 +  "thead", "tbody", "tfoot", "tr", "td", "th", "colgroup", "ca
    +ption", "rt", "rp",
507 +]);
508 +
509 +const PRESERVE_WHITESPACE = new Set(["pre", "textarea"]);
510 +
511 +// JSON.stringify is recursive and throws on very deep trees (
    +about 2800 levels in Node).
512 +// Chrome's parser does not nest deeper than 512 either; deepe
    +r elements become siblings.
513 +const MAX_DEPTH = 512;
514 +
515  function showExample1() {
516    const htmlExample = `<!DOCTYPE html>
517  <html lang="en">
```

- Частина 2 задає списки тегів: CLOSES_P для блочних тегів, IMPLICITLY_CLOSES для того, кого закриває кожен тег, і OPTIONAL_END_TAGS для елементів, чий end-тег необов'язковий. MAX_DEPTH = 512 я взяв з пам'яті про Blink, тож варто перевірити перед рев'ю. Далі переходжу до tree builder.

- Update(html2json.js)
  └ Added 181 lines

```
    512  // Chrome's parser does not nest deeper than 512 either; deepe
         r elements become siblings.
    513  const MAX_DEPTH = 512;
    514
    515 +/*
    516 +  Builds the tree from tokens with a stack of open elements, w
        +ithout recursion.
    517 +  The top of the stack is the current element: new nodes go in
        +to its children.
    518 +  Each stack entry keeps the node and what is needed while it
        +is open:
    519 +  `name` (lowercased, for matching end tags), `namespace` ("ht
        +ml", "svg" or "math"),
    520 +  `preserveWhitespace` and `start` (for warnings).
    521 +*/
    522 +class TreeBuilder {
    523 +  constructor(warnings) {
    524 +    this.warnings = warnings;
    525 +    this.root = { type: "root", children: [] };
    526 +    this.stack = [{ node: this.root, name: "", namespace: "htm
        +l", preserveWhitespace: false, start: 0 }];
    527 +    // Elements past MAX_DEPTH that are not on the stack, by n
        +ame, so their end tags are not "unexpected".
    528 +    this.flattenedOpen = new Map();
    529 +    this.depthWarned = false;
    530 +    // Browsers drop a line break right after <pre> and <texta
        +rea>.
    531 +    this.dropLeadingNewlineIn = null;
    532 +  }
    533 +
    534 +  get current() {
    535 +    return this.stack[this.stack.length - 1];
    536 +  }
    537 +
    538 +  add(token) {
    539 +    const dropLeadingNewlineIn = this.dropLeadingNewlineIn;
    540 +    this.dropLeadingNewlineIn = null;
    541 +
    542 +    switch (token.type) {
    543 +      case "startTag":
    544 +        this.startTag(token);
    545 +        break;
    546 +      case "endTag":
    547 +        this.endTag(token);
    548 +        break;
    549 +      case "text":
    550 +        this.text(token, dropLeadingNewlineIn === this.current
        +.node);
    551 +        break;
```

```
552 +      case "comment":
553 +      case "doctype":
554 +        this.append({ type: token.type, content: token.content
    + });
555 +        break;
556 +    }
557 +  }
558 +
559 +  finish() {
560 +    while (this.stack.length > 1) this.pop("is not closed befo
    +re the end of input");
561 +    return this.root;
562 +  }
563 +
564 +  startTag(token) {
565 +    const name = token.name.toLowerCase();
566 +    if (contextNamespace(this.current) === "html") this.closeI
    +mplicitly(name);
567 +
568 +    const parent = this.current;
569 +    const namespace = elementNamespace(parent, name);
570 +    const isHtml = namespace === "html";
571 +    const element = {
572 +      type: "element",
573 +      // SVG and MathML are case-sensitive (viewBox), so their
    + names are kept as written.
574 +      tag: isHtml ? name : token.name,
575 +      attributes: this.collectAttributes(token, isHtml),
576 +      children: [],
577 +    };
578 +    this.append(element);
579 +
580 +    if (token.selfClosing || (isHtml && VOID_ELEMENTS.has(name
    +))) return;
581 +    if (this.stack.length > MAX_DEPTH) {
582 +      this.flatten(name, token);
583 +      return;
584 +    }
585 +
586 +    this.stack.push({
587 +      node: element,
588 +      name,
589 +      namespace,
590 +      preserveWhitespace: parent.preserveWhitespace || PRESERV
    +E_WHITESPACE.has(name),
591 +      start: token.start,
592 +    });
593 +    if (PRESERVE_WHITESPACE.has(name)) this.dropLeadingNewline
    +In = element;
594 +  }
595 +
596 +  endTag(token) {
597 +    const name = token.name.toLowerCase();
598 +    if (token.attributes.length > 0) {
599 +      this.warn(`End tag </${token.name}> has attributes, they
    + are ignored`, token.start);
600 +    }
601 +
602 +    const flattened = this.flattenedOpen.get(name);
603 +    if (flattened) {
604 +      this.flattenedOpen.set(name, flattened - 1);
605 +      return;
606 +    }
607 +
608 +    let index = this.stack.length - 1;
609 +    while (index > 0 && this.stack[index].name !== name) index
    +--;
610 +    if (index === 0) {
611 +      this.warn(`Unexpected end tag </${token.name}> is ignore
    +d`, token.start);
612 +      return;
613 +    }
614 +
615 +    while (this.stack.length - 1 > index) this.pop(`is closed
    +by </${token.name}> before its own end tag`);
616 +    this.stack.pop();
```

```
617 +  }
618 +
619 +  text(token, dropLeadingNewline) {
620 +    if (token.formattingWhitespace && !this.current.preserveWh
    +itespace) return;
621 +
622 +    let content = token.content;
623 +    if (dropLeadingNewline && content[0] === "\n") content = c
    +ontent.slice(1);
624 +    if (content === "") return;
625 +
626 +    const children = this.current.node.children;
627 +    const last = children[children.length - 1];
628 +    if (last !== undefined && last.type === "text") {
629 +      last.content += content;
630 +    } else {
631 +      children.push({ type: "text", content });
632 +    }
633 +  }
634 +
635 +  // <p>a<div>, <li>a<li>b, <td>1<td>2: closes elements whose
    +end tag the author left out.
636 +  closeImplicitly(name) {
637 +    if (CLOSES_P.has(name) && this.current.name === "p") this.
    +pop();
638 +    const closes = IMPLICITLY_CLOSES.get(name);
639 +    while (closes !== undefined && closes.has(this.current.nam
    +e)) this.pop();
640 +  }
641 +
642 +  // First attribute wins on duplicates. A prototype-less obje
    +ct keeps names like
643 +  // "__proto__" and "constructor" as plain keys.
644 +  collectAttributes(token, isHtml) {
645 +    const attributes = Object.create(null);
646 +    for (const { name, value } of token.attributes) {
647 +      const key = isHtml ? name.toLowerCase() : name;
648 +      if (key in attributes) {
649 +        this.warn(`Duplicate attribute "${key}" on <${token.na
    +me}> is ignored`, token.start);
650 +      } else {
651 +        attributes[key] = value;
652 +      }
653 +    }
654 +    return attributes;
655 +  }
656 +
657 +  flatten(name, token) {
658 +    this.flattenedOpen.set(name, (this.flattenedOpen.get(name)
    + || 0) + 1);
659 +    if (!this.depthWarned) {
660 +      this.depthWarned = true;
661 +      this.warn(`Nesting deeper than ${MAX_DEPTH} levels is fl
    +attened: deeper elements are added as siblings`, token.start);
662 +    }
663 +  }
664 +
665 +  append(node) {
666 +    this.current.node.children.push(node);
667 +  }
668 +
669 +  // Closes the current element. `reason` becomes a warning, u
    +nless the element's
670 +  // end tag is optional (<p>, <li>, ...).
671 +  pop(reason) {
672 +    const frame = this.stack.pop();
673 +    if (reason !== undefined && !OPTIONAL_END_TAGS.has(frame.n
    +ame)) {
674 +      this.warn(`Element <${frame.node.tag}> ${reason}`, frame
    +.start);
675 +    }
676 +  }
677 +
678 +  warn(message, offset) {
679 +    this.warnings.push({ message, offset });
680 +  }
```

```
681 +}
682 +
683 +// Namespace that children of `frame` start in. Inside <foreig
    +nObject> SVG hosts HTML again.
684 +function contextNamespace(frame) {
685 +  return frame.namespace === "svg" && frame.name === "foreigno
    +bject" ? "html" : frame.namespace;
686 +}
687 +
688 +function elementNamespace(parent, name) {
689 +  const namespace = contextNamespace(parent);
690 +  if (namespace !== "html") return namespace;
691 +  if (name === "svg") return "svg";
692 +  if (name === "math") return "math";
693 +  return "html";
694 +}
695 +
696  function showExample1() {
697    const htmlExample = `<!DOCTYPE html>
698  <html lang="en">
```

● Що робить частина 3:
- Елемент стеку зберігає не лише вузол, а й «службову» інформацію, яка потрібна, поки елемент відкритий: ім'я в нижньому регістрі для пошуку end-тегу, namespace, preserveWhitespace, start. У вихідний JSON вона не потрапляє, тому дерево лишається чистим, як і з parent.
- add розподіляє токени за типом. Прапорець «викинути перший \n» живе рівно один токен: якщо одразу після <pre> іде не текст, він скидається.
- startTag:
  a. Неявне закриття, тільки в HTML-контексті.
  b. Визначаємо namespace.
  c. Створюємо вузол з атрибутами.
  d. Додаємо його в поточний елемент.
  e. Void і <x/> у стек не кладемо. На глибині понад 512 теж: такий елемент «сплющується».
- endTag шукає ім'я в стеку зверху вниз. Не знайшов — ігнорує тег з warning. Знайшов нижче вершини — закриває все, що вище, з warning для кожного (крім <p>/<li>…), і сам елемент. Перед цим перевіряє, чи це не end-тег «сплющеного» елемента.
- text: викидає відступи (окрім <pre>/<textarea>), прибирає перший \n після <pre>/<textarea>, зливає з попереднім текстом.
- closeImplicitly: спершу <p>, потім решта правил. Порядок важливий: у <li><p>текст<li> новий <li> спершу закриває <p>, а потім попередній <li>.
- collectAttributes: Object.create(null), нижній регістр тільки для HTML, при дублікаті перемагає перший + warning.
- contextNamespace / elementNamespace: всередині <svg> і <math> регістр зберігається. <foreignObject> знову вміщує HTML, як у браузері, і це покриває кейс з 16-svg-math.html.

Частина 4: html2json і line/column

Read 1 file (ctrl+o to expand)
└ Found 3 new diagnostic issues in 1 file (ctrl+o to expand)

● Update(html2json.js)
└ Added 56 lines, removed 7 lines

```
 5    jsonArea.textContent = JSON.stringify(jsonObj, null, 2);
 6  }
 7
 8 -/*
 9 -  Update this function to convert html into json object.
10 -  You can rewrite it completely, just be sure it accepts htmlTe
   -xt as string and outputs json object.
 8 +/*
 9 +  Converts an HTML string into a JSON tree:
10 +    { type: "root", children: [...], warnings: [{ message, line
   +, column }] }
11 +  Never throws: problems in the input are reported in `warnings
   +`.
12 +  See README.md for the format and the design decisions.
13  */
14  function html2json(htmlText) {
13 -  return {
14 -    "Conversion results": "should be instead of this json obj",
15 -    "Just to show that it is dynamic value (input length)" : ht
   -mlText.length,
```

```
16 -  };
15 +  if (typeof htmlText !== "string") {
16 +    const received = htmlText === null ? "null" : typeof htmlTe
   +xt;
17 +    return {
18 +      type: "root",
19 +      children: [],
20 +      warnings: [{ message: `Expected a string, got ${received}
   +`, line: 1, column: 1 }],
21 +    };
22 +  }
23 +
24 +  const input = normalizeInput(htmlText);
25 +  const warnings = [];
26 +  const builder = new TreeBuilder(warnings);
27 +  for (const token of tokenize(input, warnings)) builder.add(to
   +ken);
28 +
29 +  const root = builder.finish();
30 +  root.warnings = locateWarnings(input, warnings);
31 +  return root;
32  }
33
34 +/*
35 +  Turns { message, offset } into { message, line, column }, sor
   +ted by position.
36 +  Line starts are collected once, then each offset is found by
   +binary search.
37 +*/
38 +function locateWarnings(input, warnings) {
39 +  if (warnings.length === 0) return [];
40 +
41 +  const lineStarts = [0];
42 +  for (let i = input.indexOf("\n"); i !== -1; i = input.indexOf
   +("\n", i + 1)) lineStarts.push(i + 1);
43 +
44 +  return warnings
45 +    .slice()
46 +    .sort((a, b) => a.offset - b.offset)
47 +    .map(({ message, offset }) => {
48 +      const lineIndex = findLineIndex(lineStarts, offset);
49 +      return { message, line: lineIndex + 1, column: offset - l
   +ineStarts[lineIndex] + 1 };
50 +    });
51 +}
52 +
53 +// Index of the last line start that is <= offset.
54 +function findLineIndex(lineStarts, offset) {
55 +  let low = 0;
56 +  let high = lineStarts.length - 1;
57 +  while (low < high) {
58 +    const middle = Math.ceil((low + high) / 2);
59 +    if (lineStarts[middle] <= offset) {
60 +      low = middle;
61 +    } else {
62 +      high = middle - 1;
63 +    }
64 +  }
65 +  return low;
66 +}
67 +
68  /* ---------- Tokenizer ---------- */
69
70  // No tags inside: everything up to the matching end tag is tex
    t.
```

Ran 1 shell command (ctrl+o to expand)
└ Found 2 new diagnostic issues in 1 file (ctrl+o to expand)

● Точкові тести all passed as expected. Next I'm running all the samples together through the real JSON.stringify, the same step the Convert button performs.

Ran 1 shell command (ctrl+o to expand)

● Нічого не зламалося, але 22-deep-nesting.html дає 118 МБ JSON, тож спершу

перевірю ключові семпли, а з цією проблемою розберуся далі.

Ran 1 shell command (ctrl+o to expand)

- Тепер бачу, що два warnings для обрізаних тегів дублюють один одного на тій самій позиції, тож прибираю зайве. Далі перевірю, чи це не зачепить інші випадки.

Ran 1 shell command (ctrl+o to expand)
└ Found 2 new diagnostic issues in 1 file (ctrl+o to expand)

- Тепер README, бо на етапі 10 з'явилося кілька нових рішень:
Ran 2 shell commands (ctrl+o to expand)

- Один пункт скрипта нічого не змінював, тож скрипт зупинився до запису, а файл лишився незміненим. Я вилучаю цей пункт і запускаю його знову.

Ran 1 shell command (ctrl+o to expand)

- Етап 10 готовий: парсер повністю працює, і кнопка Convert в index.html тепер показує справжнє дерево. Зараз ми на 10 з 15 етапів, приблизно 70%. Коміт ще не робив: скажи, коли зберегти.

Що перевірив

Усі 23 семпли проходять html2json + JSON.stringify(r, null, 2) без падінь, тобто те саме, що робить кнопка Convert. Точкові кейси:

| Вхід | Результат |
|---|---|
| x | і всередині b; 2 warnings: 1:4 (і закрито через ), 1:12 (зайвий ) |
| first second | три сусіди, без warnings, бо end-тег p необов'язковий |
| вкладені <ul><li> | внутрішній <li> не закрив зовнішній |
| <tr><td>1<td>2<tr><th> | два рядки, клітинки на місці |
| Hello world | пробіл " " між ними зберігся |
| 
 /   | обидва вузли збережені |
| <pre>\n  keep</pre> | "  keep\n": перший \n викинутий, відступ лишився |
| <DIV CLASS=a class=b Id=c> | div {class: "a", id: "c"} + warning про дублікат |
| SVG + <foreignObject><DIV ID=x> | viewBox, linearGradient у регістрі як написано; div/id всередині foreignObject у нижньому |
| __proto__="x" constructor="y" | обидва атрибути на місці |
| null, undefined, 42, {} | порожній root + warning, без падіння |
| 21-bom-crlf | BOM і \r прибрані |

У 10-malformed 6 warnings, і кожен з правильним line:column.

Що ще змінилось

- Прибрав дубль warning. Для обрізаного тегу (<div class="a в кінці файлу) раніше було три warnings, тепер два: про обрізаний тег і про незакриту лапку. «Не закритий наприкінці» дублював перший.
- README: ліміт глибини, перший \n після <pre>, перевірка пробілів до декодування, foreignObject. Також там тепер є попередження, що в attributes немає методу hasOwnProperty, бо в об'єкта немає прототипу. Це зворотний бік Object.create(null), і на рев'ю це можуть спитати. У «Known limitations» додав три пункти: зайвий , </br> і text .

На що варто звернути увагу

1. Текст у <li> без закриваючого тегу має «хвіст»: "one\n  ". Це правильно за нашим правилом «текст не обрізаємо», і браузер теж зберігає ці символи. Але на вигляд негарно. Залишаємо, чи обговоримо?
2. 22-deep-nesting у Convert. Падіння вже немає, але відформатований JSON важить 116 МБ (без форматування — 595 КБ). Майже весь обсяг — відступи: ~9500 «сплющених»  лежать на глибині 512, і кожен рядок починається з ~2000 пробілів. Поки нічого не міняв, бо спершу варто перевірити, як із цим впорається textarea в реальному браузері. Це етап 12. Якщо сторінка зависне, вирішуватимемо там: скажімо, менший ліміт або компактний JSON для дуже великих результатів.

Можеш уже відкрити index.html і натиснути Convert на прикладах 1 і 2. Наступний етап 11: оформлений тест-скрипт з перевіркою очікуваних результатів, а не лише «не впало», і пошук багів.

✻ Cooked for 5m 59s · done 7:27 PM

❯ відкрив index.html у firefox і потестив.

example 2:div з трьома дітьми p, button, textarea, пробілів між ними нема, warnings пустий. все як треба. тільки довгий текст з textarea в правому полі вилазить вбік, треба скролити.це через white-space: pre в css? чіпати index.html не будемо, так?
example 1: теж норм, © в футері став ©
<ul><li>a<li>b</ul>: вийшло два окремих li всередині ul, warnings пустий. другий li сам закрив перший, як і треба

по твоїх питаннях:
1. хвіст в li лишаємо, раз браузер теж так робить. але запиши в README, шоб рев'юер не подумав шо це баг
2. ок, deep-nesting давай на етапі 12. а чому без форматування 595кб а з ним 116мб? це просто через відступи?

і пару питань:
- чому ліміт саме 512? це просто як у chrome?
- try/catch в convertHtml2JsonAndSet все одно не додати на всякий випадок? в README ж написано шо взагалі не має крашитись
- я правильно поняв стек: відкриваючий тег кладемо наверх, закриваючий знімаємо, а якщо закриваючий не той шо на вершині, то шукаємо його нижче і закриваємо все шо над ним з warning?

і коміть етап 10

Ran 1 shell command (ctrl+o to expand)

● Коміт f290e20 зроблено, разом з ним README з поясненням про «хвіст» у <li>. Тобто етап 10 збережено, і ми все ще на 10 з 15, приблизно 70%. Тепер відповіді.

Довгий текст у правому полі

Так, причина саме в white-space: pre для textarea в CSS index.html. Цей стиль забороняє перенос рядків, тож рядок "content": "Some very very ..." стоїть в одному рядку й вилазить вбік. JSON при цьому правильний: JSON.stringify і має писати рядок в один рядок.

index.html не чіпаємо. README дозволяє лишити його без змін («You can leave it unchanged»), а сторінку дали нам як оболонку для перевірки. Змінювати її вигляд — не наше завдання.

Чому 595 КБ проти 116 МБ

Так, це відступи. Рахунок такий:
- 9 488 «сплющених»  лежать на глибині 513 елементів
- кожен елемент дає в JSON два рівні вкладеності: об'єкт {} і масив children: []. Тобто це ~1 026 рівнів × 2 пробіли ≈ 2 050 пробілів на початку кожного рядка
- порожній  у форматованому JSON займає ~6 рядків ({, "type", "tag", "attributes", "children", }), тобто ≈ 12 КБ, з яких корисних даних менше ніж 100 байт
- 9 488 × 12 КБ ≈ 115 МБ

Без форматування відступів немає взагалі, і той самий  займає ~60 байт.

Чому саме 512

Ліміт має бути значно нижчим за ~2800, де падає JSON.stringify у Node. В інших рушіях поріг інший: ти тестуєш у Firefox, і там треба перевірити окремо, це буде на етапі 12. З іншого боку, справжні сторінки рідко бувають глибшими за кілька десятків рівнів, тож будь-яке число від ~200 до ~1000 нормальної HTML не зачепить.

Тобто конкретне число довільне. 512 я взяв, бо його використовує відомий парсер (Chrome), і це легше обґрунтувати, ніж «я так вирішив». Пояснювати на рев'ю варто саме так: межа потрібна через рекурсивний JSON.stringify, а значення взяте з Chrome і має запас в обидва боки.

try/catch

Я б розділив, де його ставити:
- Усередині html2json не ставив би. Відомих способів його зламати ми не знаємо, і всі семпли проходять. Якщо там з'явиться баг, try/catch перетворить падіння на тихий порожній результат, тобто сховає баг.
- У convertHtml2JsonAndSet поставив би. Там є місце, яке від парсера не залежить: JSON.stringify для дуже великого результату може кинути RangeError: Invalid string length. Ми це бачили на 116 МБ, а V8 обмежує рядок приблизно 512 МБ–1 ГБ. Без try/catch кнопка просто нічого не зробить, а помилка буде видна лише в консолі. З try/catch у правому полі з'явиться зрозуміле повідомлення.

Це та сама функція, яку ми чіпатимемо через 22-deep-nesting, тож пропоную зробити обидві речі разом на етапі 12. Ще раз: сам index.html не змінюється, convertHtml2JsonAndSet лежить у нашому html2json.js.

Стек

Так, ти зрозумів правильно. Ще кілька деталей, які на рев'ю краще згадати самому:
1. Start-тег може спершу зняти щось зі стеку, ще до того, як покласти себе. Це неявне закриття: <li> знімає попередній <li>, якщо той на вершині.
2. Void-теги й <x/> у стек не кладуться взагалі, бо в них не буде дітей.
3. Закриваючого тегу немає в стеку зовсім: тег ігнорується, warning.
4. Знайшли нижче вершини: усе, що над ним, знімаємо. Warning лише для тих, у кого end-тег обов'язковий, тож text закриє  мовчки, бо  можна не писати.
5. Кінець входу: знімаємо все, що лишилось, за тим самим правилом warnings.
6. Глибина понад 512: елемент не кладемо в стек, а його end-тег потім тихо «з'їдаємо».

Наступний етап 11: тест-скрипт. Пропоную tests/run.js для Node:
- на кожному семплі перевіряє, що нічого не падає, JSON.stringify проходить, а час у межах норми
- точкові тести з очікуваним результатом, тобто з перевіркою конкретного дерева і warnings, а не лише «не впало»
- кейси, які ми розібрали в розмові (
, x, __proto__ тощо), перетворюємо на тести, щоб вони не зламалися при наступних змінах

Тести я б поклав в окрему папку tests/. У README її немає в «Expected repository structure», але структура там не забороняє нічого додавати, а тест-скрипт тільки посилить роботу. Почнемо?

✻ Cogitated for 44s · done 7:51 PM

※ recap: Ми пишемо власний HTML→JSON парсер для тестового завдання Jito. Парсер уже готовий і закомічений, це етап 10 з 15, приблизно 70%. Далі етап 11: тест-скрипт у `tests/`, чекаю твого «так», щоб почати.

❯ ок, давай етап 11, tests/ норм, пояснення по try/catch зрозумів, логічно. давай на етапі 12.

етап 11, tests/ норм. пару питань:
- html2json.js це ж файл для браузера, там є document. як node його підключить?
- чому без jest чи якоїсь бібліотеки для тестів? просто на node норм?
- tests/ теж кладемо в архів?
- і поясни як я сам потім буду запускати тести

● Спершу відповіді.

Як Node підключить файл з document

document використовується тільки всередині тіл функцій (convertHtml2JsonAndSet, showExample1, showExample2). Коли Node завантажує файл через require, він лише оголошує ці функції, а не виконує їх. Тіло функції запуститься тільки при виклику, а тести їх не викликають. Виконується лише код верхнього рівня: константи, оголошення функцій і класу, і блок if (typeof module !== "undefined") module.exports = ... в кінці. Саме для цього блок там і стоїть.

Чому без jest

Тут потрібна бібліотека, і вона вбудована в Node: node:test (раннер з describe/test) і node:assert (перевірки) є в Node з версії 18. Переваги:
- рев'юеру нічого не треба встановлювати: ні package.json, ні npm install, ні папки node_modules на сотні МБ в архіві
- менше рухомих частин — менше шансів, що в рев'юера «не запуститься». У README прямо сказано, що непрацюючі частини означають провал

Jest дав би кращий вивід і mock-функції, але для чистої функції «рядок на вході, об'єкт на виході» вони не потрібні.

Чи класти tests/ в архів

Так. Тести показують, як ти перевіряв роботу, а це прямо входить в оцінку («test your code again to ensure it still functions correctly»). У README додам розділ «Running tests».

Як запускати

З кореня проєкту:
node --test
Node сам знайде файли *.test.js і покаже, скільки тестів пройшло. Потрібен Node 18+, у тебе 26. Один файл можна запустити й напряму: node tests/html2json.test.js.

---

Що буде в тестах

1. Точкові тести з очікуваним деревом для кожної групи рішень: теги, атрибути, текст, entities, raw text, коментарі, неявне закриття, невалідний HTML, SVG, warnings з line:column.
2. Кожен семпл з html_samples/: не падає, JSON.stringify проходить, дерево проходить перевірку інваріантів.
3. Інваріанти, тобто правила, які мають виконуватися для будь-якого результату:
   - у кожного елемента є tag, attributes, children
   - два text-вузли ніколи не стоять поруч
   - текст ніколи не порожній
   - глибина ≤ 513
   - у кожного warning є line і column ≥ 1
4. Фазинг (fuzzing): 5000 випадкових рядків, зібраних зі шматків HTML (<, </, <!--, &#, ", script, svg…). Перевіряємо, що нічого не падає і інваріанти виконуються. Генератор детермінований (з фіксованим seed), тож якщо тест упаде, той самий рядок можна відтворити.
5. Патологічні входи на швидкодію: мільйон &, мільйон <, 100 000 зайвих  на глибокому стеку тощо. Саме на таких входах наївна реалізація стала б O(n²).

Один нюанс: порівнюю результат після JSON.parse(JSON.stringify(...)), а не напряму. По-перше, саме в такому вигляді результат використовують. По-друге, assert.deepStrictEqual порівнює й прототипи, тож attributes без прототипу ніколи не дорівнювали б звичайному {} в очікуваному значенні.

● Write(tests/html2json.test.js)
  ⎿  Wrote 593 lines to tests/html2json.test.js
     1 // Run from the project root with: node --test
     2 const { describe, test } = require("node:test");
     3 const assert = require("node:assert/strict");
     4 const fs = require("node:fs");
     5 const path = require("node:path");
     6 const { html2json } = require("../html2json.js");
     7
     8 const SAMPLES_DIR = path.join(__dirname, "..", "html_samples");
     9 const MAX_ELEMENT_DEPTH = 513; // MAX_DEPTH levels plus the flatt

ened elements on top
… +699 lines (ctrl+o to expand)

Ran 1 shell command (ctrl+o to expand)

- Пройшло 99 із 101 тестів, і обидва падіння — помилки в самих тестах, а не в парсері. Парсер у першому випадку правий, а в другому я на одиницю помилився з кількістю кроків; далі виправлю тест і додам перевірки на 512 і 513 рівнях.

  Ran 1 shell command (ctrl+o to expand)

- Усі 102 тести тепер проходять. Далі я перевірю, чи вони справді ловлять помилки: зроблю копію проєкту в scratchpad і по черзі внесу три навмисні баги.

  Ran 1 shell command (ctrl+o to expand)

- Два з трьох тестів упали через помилки, а третій вижив. Прибирання обмеження в readNumericReference нічого не ламає, бо Infinity > MAX_CODE_POINT все одно дає U+FFFD, тож справжній захист — це перевірка перед String.fromCodePoint. Я залишу обмеження для читабельності, але виправлю коментар і README, які подавали його як обов'язкове.

  Ran 1 shell command (ctrl+o to expand)

- Етап 11 готовий: усі 102 тести проходять за ~0,6 с. Зараз ми на 11 з 15 етапів, приблизно 78%. Коміт ще не робив: скажи, коли зберегти.

  Що в tests/html2json.test.js

| Група | Що перевіряє |
|---|---|
| output format | example 2 повністю, кілька коренів, порожній вхід, не рядок на вході |
| tags, attributes | void, , регістр, усі види лапок, дублікати, __proto__/constructor |
| text and whitespace | trim, відступи, " " між inline-тегами, 
, <pre>, хвіст у <li> |
| character references | іменовані, числові, криві числа, Windows-1252, старі імена без ;, URL в атрибутах |
| raw text, comments | script/style/textarea/title, "</script>" у JS, усі форми коментарів, bogus |
| implicit closing, invalid HTML | p/li/tr/td/option/dt, неправильна вкладеність, обриви, зайві теги |
| svg, warnings | регістр у SVG, foreignObject, точні line:column, \r\n, BOM, сортування |
| depth limit | 10 000 рівнів; рівно 512 рівнів без змін; 513-й стає сусідом |
| html_samples | кожен з 23 файлів + інваріанти |
| robustness | 5000 випадкових входів + 9 патологічних на швидкодію |

Що знайшли тести

- У парсері багів не знайдено. Два перші падіння виявилися помилками в самих тестах: неправильне очікування для b" і помилка на одиницю в лічильнику глибини. Я виправив тести, парсер не чіпав.
- Перевірка, чи тести ловлять баги. Я тричі навмисно ламав код:
  - перевірка пробілів після декодування → тест упав ✔
  - атрибути в звичайному {} → тест упав ✔
  - прибраний cap у числах → тест пройшов. Але це не діра в тестах: cap справді не потрібен для правильності, бо великі числа в JS стають Infinity, а не «перекручуються», і перевірка > MAX_CODE_POINT їх однаково ловить. Обґрунтування в коді й README я виправив. Тепер на питання «а якщо число величезне?» правильна відповідь така: справжній захист — це перевірка діапазону перед String.fromCodePoint.

README

Додав розділ Running tests: як запускати (node --test), що саме перевіряється і чому браузерний файл працює в Node. Папку tests/ додав у «Repository structure».

Наступний етап 12: перевірка в браузері. Пропоную такий план:
1. Ти відкриваєш у Firefox 22-deep-nesting.html і 23-large.html (вставити вміст файлу в ліве поле → Convert) і кажеш, що побачив: скільки часу зайняло, чи зависла сторінка, чи не з'явилась помилка в консолі (F12 → Console).
2. За результатами вирішуємо, що робити з 116 МБ.
3. Додаємо try/catch у convertHtml2JsonAndSet.

Якщо є Chrome, перевір і в ньому: поріг для JSON.stringify у рушіях різний.

✻ Cogitated for 3m 24s · done 8:00 PM

● Keeping auto (1m tokens) · /autocompact to change

❯ /export
  ⎿ Conversation exported to: /Users/barsik/Desktop/test work project/ai_help/claude_code_chat_backup.txt

※ recap: Ти робиш тестове html2json; етап 11 з 15 (тести, 102 з 102 проходять) готовий, але ще не закомічений. Далі відкрий у Firefox 22-deep-nesting.html і 23-large.html, натисни Convert і скажи, чи сторінка не зависає.
  ⎿ Referenced file tests/html2json.test.js
  ⎿ Referenced file html2json.js
  ⎿ Read README.md (145 lines)

● Compacted while idle, before the prompt cache expired · turn off with "idleCompaction": false in settings.json

❯ комить етап 11.

потестив у firefox:
– 23-large.html: те саме шо і для другого, помилок нема і чекав стільки ж
– 22-deep-nesting.html: помилок в консолі не було, чекав приблизно ну до 1 секунди, шось тіпа 0.5–0.8 секунди я чекав

і пару питань по тестах:
– шо таке фазинг і нащо там seed? тіпа шоб можна було повторити той самий випадковий тест?
– інваріанти я правильно поняв, це правила які мають бути правдою для будь-якого результату, не важливо який html на вході?

Ran 2 shell commands (ctrl+o to expand)

● Закомітив етап 11: 92d8410 Add tests: expected trees, samples, invariants, fuzzing and performance. Перед комітом ще раз запустив тести: 102 з 102 пройшли.

ai_help/claude_code_chat_backup.txt, який створив /export, у коміт не додавав. README просить для не-ChatGPT формат .pdf/.png/.mp4, тож один .txt не підходить. До того ж розмова ще триває, і експорт зараз був би неповним. Розберемося на етапі 14.

Твої питання

Фазинг: так, ти правильно зрозумів. Фазинг означає, що ми генеруємо тисячі випадкових вхідних даних і перевіряємо, що програма не падає і результат валідний. Руками такі комбінації не придумаєш: <a b='<!--, &#x</script, </svg/>. У нас тест склеює випадкові шматки з FUZZ_PIECES (<div, ", &amp, <!--, </, >…) у 5000 рядків.

Seed потрібен, бо Math.random() щоразу дає нові числа. Якщо тест упаде, ти не зможеш повторити той самий вхід. Тому в нас свій генератор mulberry32(20261101): він видає «випадкові» числа, але з тим самим seed послідовність завжди однакова. Звідси дві речі:
– тест детермінований: однаковий результат у тебе, в рев'юера і на CI;
– якщо щось упаде, вхід, на якому впало, можна відтворити й налагодити.

Якщо змінити seed, отримаєш нові 5000 випадків, тобто так можна «пошукати ще».

Інваріанти: так, саме так. Це правила, які мають виконуватися для будь-якого результату, хай навіть HTML на вході повна каша. Для фазингу вони незамінні: для випадкового входу ми не знаємо правильного дерева, зате знаємо, яким воно точно не може бути. У нас це такі правила:
- кожен element має tag, attributes і children;
- немає порожніх текстових вузлів і двох текстових підряд;
- глибина ≤ 512;
- у кожного warning є line і column ≥ 1;
- результат проходить через JSON.stringify.

Firefox

Результати хороші. 23-large працює так само, як example 2, а 22-deep-nesting займає 0,5–0,8 с без помилок. Отже, 116 МБ форматованого JSON Firefox витримує. Пропоную нічого з цим не робити: ліміт 512 уже захищає від справжнього краху, а 10 000 вкладених div у реальності не зустрінеш. Додам лише рядок у Known limitations, що на такому вході JSON виходить великим через відступи.

Етап 12 (≈80% → далі)

1. Додаю try/catch у convertHtml2JsonAndSet: якщо станеться щось непередбачене, повідомлення про помилку з'явиться в правому полі, а сторінка не «мовчатиме». index.html не чіпаю.
2. Рядок про розмір JSON у README.

Робити?

✻ Baked for 31s · done Sunday 9:26 PM

※ recap: Ти робиш тестове html2json; етап 11 (тести, 102/102) закомічено, далі етап 12. Наступний крок: підтвердь, щоб я додав try/catch у convertHtml2JsonAndSet і рядок про розмір JSON у README.

❯ так, давай етап 12.

ще хочу запитати, а шо саме побачить користувач в правому полі якщо щось впаде? просто текст помилки? а також CI це ти маєш на увазі типу github actions, де тести самі запускаються при пуші?

з приводу тхт файлу і те як я здам роботу з тобою не хвилюйся я запишу просто на відео теміналл нашої з тобою розмови, тхт файл чисто на всяк випадок як бекап

Ran 2 shell commands (ctrl+o to expand)

● Now I'll check that the catch actually works, using a fake document in Node, and add the README line.

Ran 2 shell commands (ctrl+o to expand)

● Етап 12 готовий, коміт поки не робив. Ми на 12 з 15, приблизно 85%.

Що змінив

convertHtml2JsonAndSet тепер обгорнута в try/catch:
```
try {
  const jsonObj = html2json(htmlTextAreaValue);
  jsonArea.textContent = JSON.stringify(jsonObj, null, 2);
} catch (error) {
  console.error(error);
  jsonArea.textContent = `Conversion failed: ${error}`;
}
```
У try сидять обидва виклики, бо впасти може кожен: html2json через баг, а JSON.stringify через глибину чи пам'ять. index.html я не чіпав.

README: додав два рядки:
- у Robustness: що бачить користувач, коли щось падає;
- у Known limitations: що глибокий вхід дає ~116 МБ форматованого JSON (а без відступів ~600 КБ) і що Firefox рендерить його менш ніж за секунду.

Перевірка: у браузері помилку викликати не вийде, бо парсер не падає. Тому я підсунув сторінці в Node фейковий document і JSON.stringify, який навмисно кидає помилку. Звичайний вхід дав нормальний JSON, а в правому полі з'явився текст Conversion failed: RangeError: Maximum call stack size exceeded. Усі 102 тести так само проходять.

Твої питання

Що побачить користувач? Один рядок: Conversion failed:, потім тип помилки і повідомлення, наприклад RangeError: Maximum call stack size exceeded. Шаблон ${error} викликає error.toString(), а він дає саме «Тип: повідомлення». Повний stack trace з файлом і номером рядка йде в консоль (F12), бо звичайному користувачу він ні до чого, а розробнику потрібен. Без try/catch праве поле лишилося б старим або порожнім, і користувач не зрозумів би, що сталося: помилка була б лише в консолі.

CI? Так, саме це: GitHub Actions чи щось подібне, де на кожен push або PR сервер сам запускає node --test і показує ✅ або ❌. Через seed фазинг дає там той самий результат, що й у тебе. Для тестового завдання CI не обов'язковий. Якщо хочеш, можу додати файл .github/workflows/test.yml на ~10 рядків, але він запрацює, тільки якщо репо буде на GitHub.

Відео: ок, тоді на етапі 14 нічого не експортую. Лишилось вирішити з .txt: він зараз лежить в ai_help/ поза git. Можу додати його в .gitignore, щоб він випадково не потрапив у коміт чи архів, або залишити як є.

Комітити етап 12? Після цього етап 13: чистка коду і фінальна вичитка README.

✻ Sautéed for 45s · done Sunday 9:35 PM

❯ так короче давай коміть етап 12
CI не треба, ми ж здаємо архів, а не репо на гітхабі, тож він там ніколи не запуститься
по .txt: в gitignore не треба. я думаю на етапі 14 зробимо фінальний /export і перетворимо його в pdf, бо в README для інших AI просять .pdf/.png/.mp4. тоді в ai_help буде і відео, і pdf з усією розмовою, pdf ж легше читати ніж годинне відео. норм ідея?

і давай етап 13

Ran 1 shell command (ctrl+o to expand)

● Stage 13: I'll read the whole file for cleanup.

Read 1 file (ctrl+o to expand)

● Reviewed the whole file. Now I'll check mechanically for unused code, leftovers and long lines.

Ran 1 shell command (ctrl+o to expand)

● Знайшов кілька дрібниць: UI-функції, які вважалися невикористаними, викликаються з index.html, а експорт tokenize тести не використовують. Тепер прибираю його.

Ran 3 shell commands (ctrl+o to expand)

● Етап 12 закомічено (83fd6d2), етап 13 зроблено, але ще не закомічено. Ми на 13 з 15, приблизно 90%.

Етап 13: що зробив

Перечитав html2json.js повністю, від початку до кінця, і перевірив його скриптом на невикористаний код, забуті console.log/TODO і надто довгі рядки. Серйозних проблем не знайшов, лише дрібниці:
- Прибрав tokenize з module.exports. Він лишився з етапу 7, коли я тестував токенізатор окремо, а тести його вже не імпортують. Тепер назовні видно тільки html2json, тобто рівно те, що просить завдання.
- Прибрав warnings = [] за замовчуванням у tokenize. Це теж залишок тих ранніх тестів, бо тепер функцію викликає лише html2json, і завжди зі своїм масивом.
- Додав cutOff: false у кореневий елемент стеку, щоб усі елементи стеку мали однакові поля, як описано в коментарі класу.
- Розбив один рядок на 125 символів.

Скрипт показав convertHtml2JsonAndSet і showExample1/2 як «невикористані», але насправді їх викликають кнопки в index.html. Довгі рядки в showExample* оригінальні, їх я не чіпав. Довгі рядки в тестах — це вхідний HTML разом з очікуваним деревом, і розбивати їх тільки гірше для читання.

Перевірка: 102 з 102 тестів пройшли, line endings у файлі лишились CRLF. Усі 23 семпли проходять без винятків: 23-large за ~50 мс, 22-deep за ~14 мс.

README перечитав, він збігається з кодом, правити нічого не треба.

Ідея з PDF

Так, норм: відео підтверджує, що розмова справжня, а PDF зручно читати й шукати в ньому. Перевірив твій поточний бекап: там 3500 рядків, і він починається з твого першого повідомлення («привіт в мене тестове…»). Отже, /export зберігає всю розмову, а не лише частину після стиснення контексту.

На етапі 14 ти робиш фінальний /export, а я конвертую його в PDF. Питання для тебе: .txt залишаємо в ai_help/ поряд з PDF чи видаляємо? Відео, мабуть, буде великим. Якщо воно не влізе в архів, README дозволяє дати посилання, але тоді треба перевірити, що воно відкривається в режимі інкогніто.

Комітити етап 13?

✻ Cogitated for 1m 2s · done Sunday 9:39 PM

※ recap: Ми робимо тестове html2json для Jito, зараз завершили етап 13 (чистка коду, 102/102 тести проходять), але ще не закомітили. Далі: підтверди коміт етапу 13 і скажи, чи залишати .txt поруч із PDF.

❯ а шо значить crlf? це ж через windows, там переноси рядків \r\n замість \n? нормально шо так лишили, чи краще зробити lf?

коміть етап 13

з приводу txt файлу поки хз, не чіпай, вирішу в самому кінці вже коливсе буде готовим, відео стисну, а якщо буде завелике, залию на google drive або youtube як unlisted і покладу посилання в ai_help. потім перевірю в інкогніто. короче за це все не думай зараз, як саме я здам проект вирішу в самому кінці коли з ним завершу.

давай етап 14

Ran 1 shell command (ctrl+o to expand)

● Закомітив етап 13 як 4f2193f. Тепер перевірю, чи вміє cupsfilter, вбудований у macOS, робити PDF з кирилицею.

Read 1 file, ran 5 shell commands (ctrl+o to expand)

● Я перевіряю, чому конвертація експорту розмови в PDF через cupsfilter дала лише 9 сторінок. Далі шукатиму місце, де обривається текст.
Ran 2 shell commands (ctrl+o to expand)

● cupsfilter тихо втрачає середину файлу після рядка ~506, тож не підходить. Далі спробую інший шлях через вбудований у macOS рушій друку.

Read 1 file, ran 5 shell commands (ctrl+o to expand)

● PDF-конвертер працює. Нижче відповідь про CRLF і етап 14.

CRLF

Так, ти правильно зрозумів. CRLF означає \r\n, тобто два символи: Carriage Return («каретку на початок рядка») і Line Feed («новий рядок»). Назви лишились з друкарських машинок. Windows використовує CRLF, а macOS і Linux лише LF (\n).

git ls-files --eol показує, що в оригінальному завданні html2json.js і index.html вже були в CRLF. Схоже, автор писав їх на Windows. Наші README, тести й семпли в LF.

Раджу лишити CRLF. Якщо перевести файл у LF, кожен рядок формально зміниться. Тоді diff проти першого коміту показав би, що змінено весь файл, і рев'юер не побачив би, що саме я написав, а що було в оригіналі. Зараз diff показує лише справжні зміни. На роботу коду це ніяк не впливає: JS однаково читає \r\n і \n. Через це я після кожного редагування й повертав CRLF.

Якщо на рев'ю спитають, кажи так: «зберіг line endings оригінального файлу, щоб diff був чистим».

Коміт етапу 13: 4f2193f.

Етап 14: PDF з розмови

PDF має містити всю розмову, тому створювати його треба останнім кроком, коли ми закінчимо. Тож зараз я підготував і перевірив конвертер, а запустимо його в самому кінці.
- cupsfilter, вбудований у macOS, не підходить. На твоєму бекапі він дав 9 сторінок замість ~47 і мовчки викинув середину розмови, без жодної помилки. Добре, що я це перевірив.
- Написав Swift-скрипт через стандартний друк macOS (той самий, що «Файл → Друк → Зберегти як PDF»), без жодних встановлень. Формат A4, моноширинний шрифт, кирилиця й рамки таблиць відображаються нормально.
- Перевірка на поточному бекапі: 47 сторінок за 3 с, 114 873 символи як у .txt, так і в PDF. Різниця лише в двох місцях:
  - емодзі 🏞️ втратив невидимий модифікатор (U+FE0F), на вигляд це не впливає;
  - PDF-читалка видає довгі рядки JSON в іншому порядку, а на сторінці текст стоїть правильно, я це перевірив на зображенні.

Скрипт лежить тут: /private/tmp/claude-501/-Users-barsik-Desktop-test-work-project/2bb1d7fa-752d-4bf6-8d64-934b72ba764a/scratchpad/txt2pdf.swift.

У кінці: ти робиш фінальний /export, кажеш мені назву файлу, і я одразу кладу PDF в ai_help/. Що робити з .txt і відео, вирішуєш сам, як і домовились.

Ми на 14 з 15, приблизно 93%, нового коміту поки немає. Далі етап 15: нагадування про git author і фінальний чекліст. Можемо робити?

✻ Crunched for 2m 33s · done Sunday 9:49 PM

※ recap: Робимо тестове html2json: етапи 1–13 закомічені, а для етапу 14 PDF-конвертер розмови готовий і перевірений. Далі етап 15: вирішити з git-автором комітів і пройти фінальний чекліст, потім фінальний /export → PDF.

❯ клод привіт я випадково перезапустив ноут й все злетіло, також був зайнятий цілий день, нагадай на чому ми зупинились а будьласка ресторни історію, що в нас планах що по роадмепу на якому ми етапі що ми зараз робимо, перевірь чи все на місці все що ми робили та скажи що залишилось зробити

Ran 2 shell commands (ctrl+o to expand)

● Усе на місці, нічого не злетіло. Ми на етапі 15 з 15, приблизно 93%: код готовий і закомічений, лишилися фінальні перевірки й здача. Історія теж ціла: це та сама сесія, тож фінальний /export міститиме всю розмову. Інші сесії в цій папці, судячи з перших повідомлень, містять тільки brew upgrade claude-code, тож вимогу «все в одній розмові» це не порушує.

Що перевірив зараз

| Що | Стан |
|---|---|
| Коміти | ✅ 8 комітів, останній 4f2193f |
| Незакомічені зміни | ✅ немає, лише ai_help/claude_code_chat_backup.txt, який ти просив не чіпати |
| html2json.js | ✅ є, CRLF збережено |
| index.html | ✅ без змін відносно першого коміту |
| html_samples/ | ✅ 23 файли |
| Тести | ✅ 102 з 102 проходять |
| README.md | ✅ є |
| Скрипт для PDF txt2pdf.swift | ❌ зник: він лежав у тимчасовій папці, яку очистив перезапуск |

Скрипт був лише допоміжним і в проєкт не входив. Його код є в нашій розмові, тож я відновлю його за хвилину, коли дійде до PDF.

Роадмеп

| Етап | Стан |
|---|---|
| 1–6: аналіз, git init, edge cases, формат JSON, README, семпли | ✅ |
| 7–9: токенізатор, коментарі/doctype, entities | ✅ |
| 10: tree builder і html2json | ✅ |
| 11: тести | ✅ |
| 12: перевірка у Firefox і try/catch | ✅ |
| 13: чистка коду, вичитка README | ✅ |
| 14: PDF з розмови | ⏳ конвертер перевірено, сам PDF робимо в кінці |
| 15: git author і фінальний чекліст | ⏳ ми тут |

Що залишилось

1. Git author (твоє рішення). Ти просив нагадати, тому нагадую: усі 8 комітів мають автоматичний автор Barsik <barsik@ip-192-168-0-...tv9.home>, бо user.name і user.email не налаштовані. Раніше я пропонував виправити лише останній коміт, але цього вже мало: тепер треба переписати всю історію, і в усіх комітів зміняться хеші. Це безпечно, бо репо ніде не опубліковане, а дати авторства можна зберегти.
2. Повторна перевірка у Firefox. Після етапу 12 я змінював код, а перевіряв його лише в Node. Завдання прямо вимагає протестувати все ще раз перед здачею. Відкрий index.html, натисни Example 1 і Example 2 з Convert, потім встав 22-deep-nesting.html і 23-large.html.
3. Фінальний /export. Ти робиш його останнім, а я створюю з нього PDF в ai_help/.
4. Здача (вирішуєш сам у кінці):
   - відео або посилання на нього; посилання перевір в інкогніто, бо завдання попереджає, що з неробочими посиланнями роботу не розглядатимуть;
   - що робити з .txt;
   - архів: .DS_Store лежить у корені проєкту й git його ігнорує, але в архів він потрапить, якщо не виключити його вручну. Також реши, чи включати .git: він показує історію комітів.

Потрібне твоє рішення

Що робимо з автором комітів?
- Виправити: напиши, які ім'я та email поставити, і я перепишу всі 8 комітів.
- Лишити як є: тоді одразу переходимо до Firefox.

✻ Baked for 44s · done 2:00 PM

※ recap: Здаємо тестове html2json: код готовий і закомічений, 102 тести проходять, лишилися фінальні перевірки на етапі 15. Далі твоє рішення: виправити автора всіх 8 комітів (напиши ім'я та email) чи лишити як є. (disable recaps in /config)

❯ гаразд дивись давай зробимо так:

- автора комітів запиши як Sebastian Horhula-Kovalenko на всяк випадок, емейл робочий coldcloudyfriday@gmail.com, якщо десь мєлькав інший то це мій особистий тому прибери його вєздє де він був
- все перевірив, працює й конвертує
- виходить поки що не робити? маякни коли прям все буде закрито й треба буде лише це зробить
- чесно не знаю, по ідеї txt файл допоможе нам просто всю ашу з тобою розмову конвертувати в правильний формат здачі, оскільки я випадково перезавнтажив ноут й в мене нема всієї історії чату я не зможу здати відео формат нашої розмови, які формати ще по вимогам підходять щоб це все здати?
- DS_Store в архів не клади, .git думаю включити, щоб було видно історію комітів по етапах, вони ж оцінюють процес і додай в README рядок, що .git в архіві навмисно і чому

Ran 4 shell commands (ctrl+o to expand)

● Автора замінено в усіх 8 комітах, особистий email прибрано з усіх файлів проєкту. Ми на етапі 15, приблизно 97%.

Автор комітів

- Новий автор: усі 8 комітів тепер від Sebastian Horhula-Kovalenko <coldcloudyfriday@gmail.com>, він же прописаний у конфігу цього репо.
- Що не змінилось: вміст файлів, дати й повідомлення комітів збігаються з попередніми, тести проходять 102 з 102. Хеші нові, останній коміт тепер 0596c22.
- Старі коміти: видалив їх і з .git, щоб старий автор не потрапив в архів. Перед цим зробив копію в тимчасовій папці (/private/tmp/claude-501/-Users-barsik-Desktop-test-work-project/2bb1d7fa-752d-4bf6-8d64-934b72ba764a/scratchpad/before-author-rewrite.bundle), вона зникне при перезапуску.

Особистий email

- Де був: у комітах і файлах проєкту його не було. Він знайшовся лише в бекапі .txt, у рядку 99, де я колись показував приклад команди. Там тепер стоїть [email removed], решту бекапу не чіпав.
- Що лишилось: він є в локальному лозі цієї сесії Claude Code (~/.claude/projects/...), бо це email твого акаунта. Цей файл в архів не йде, тому я його не редагував.
- Фінальний експорт: там цей рядок з'явиться знову, тому перед створенням PDF я заміню email ще раз і перевірю результат.

Формат здачі

Для Claude завдання вимагає .pdf, .png або .mp4, тож одного PDF достатньо, відео не обов'язкове. Посилання потрібне тільки для ChatGPT.

Історія теж не втрачена: лог цієї сесії містить усі твої 17 повідомлень, від першого «привіт…» до останнього. Я ще не перевіряв, чи /export після перезапуску віддає всю розмову. Якщо початку там не буде, зберу текст напряму з логу.

Раджу покласти в ai_help/ лише PDF, а .txt прибрати перед здачею: цього формату немає в списку, і він дублює PDF.

Архів і README

- README: додав рядок, що .git/ в архіві навмисно, щоб було видно, як рішення будувалося по етапах. Ця зміна ще не закомічена.
- Архів: коли дійде до нього, зберу його з .git і без .DS_Store.

Що далі

1. Скажи «коміть», і я закомічу README. Після цього код і README будуть повністю закриті, а Firefox ти вже перевірив.
2. Це і є мій маяк: щойно коміт буде готовий, ти робиш фінальний /export і пишеш мені шлях до файлу.
3. Я перевіряю, що експорт повний, прибираю email, роблю PDF, збираю архів і перевіряю його вміст.

Комітити README?

✻ Cooked for 1m 13s · done 2:16 PM

❯ гаразд тоді давай по пунктах:

1. коміть
2. не поняв коли саме писати команду експорт? типу по серед того як ти думаєш? чи вже коли відповісиш?
3. 
   _

Ran 1 shell command (ctrl+o to expand)

● Закомітив README: 23fd1d4, автор уже новий, тести проходять 102 з 102. Код і README повністю закриті, тож можна робити експорт.

Коли писати /export: коли я вже відповів і чекаю на тебе, тобто просто зараз, після цього повідомлення. Посеред моєї роботи не треба, бо експорт може вийти неповним.

1. Введи /export і натисни Enter. Якщо він запитає, куди зберегти, обери файл у папці ai_help/.

2. Він покаже шлях до файлу, напиши його мені наступним повідомленням.

Далі я перевірю, що експорт повний, приберу email, зроблю PDF і зберу архів. Цих останніх кроків у PDF не буде, бо вони відбудуться вже після експорту. На здачу це не вплине: уся робота над завданням там є.

✻ Cogitated for 8s · done 2:20 PM